

HW4

It maybe takes two?

1. 팀원

- 배상혁 20223085

업무 경험: 게임소프트웨어에서 Unity, GitHub를 사용한 애자일 방식으로 4인 프로젝트 완성 후 배포한 경험

강점: 어떻게든 문제를 해결해내는 끈기

역할: 플레이어 구현 및 UI, 씬 관리

- 이규원 20220611

업무 경험: 게임 소프트웨어에서 팀장을 맡아 애자일 및 칸반 형식을 사용한 4인 팀 게임 프로젝트 경험

강점: 한번 맡은 일은 확실하게 처리하고 끝낸다는 책임감

역할: 게임에서의 멀티 구현 및 전반적인 서버 담당

- 양준영 20223104

업무 경험: 소프트웨어프로젝트2에서 4인 팀 프로젝트로 GitHub, Pygame을 이용한 게임 개발 경험

강점: 문제 해결을 위해 자료를 찾고 선별하는 능력

역할: 스테이지 기믹 및 상호작용 가능한 물체 구현

2. Vision & Scope

Unity & Photon: 안정적이고 끊임 없는 2인 멀티플레이 환경 구축

Co-op Puzzle: 유기적인 협동이 필요한 창의적인 퍼즐 레벨 디자인

핵심 시스템 설계 및 구현

- 캐릭터 컨트롤: 3D 기반 이동, 점프 및 물리 기반 조작
- 퍼즐 기믹: 오브젝트 상호작용(잡기, 밀기) 및 트리거 시스템
- UI/UX: 멀티플레이 로비, 서버 접속 및 인게임 인터페이스

멀티플레이어 환경 (2인 협동)

- Photon 엔진: 실시간 동기화(위치, 애니메이션, 오브젝트 상태)
- 협동 메커니즘: 2인 협력이 필수적인 퍼즐 로직 구현
- 게임 흐름: [접속 → 매칭 → 스테이지 진행 → 클리어] 프로세스 구축

Feasibility

Before 단계에서의 feasibility는 핵심 기능 중심의 범위 제한, Unity/Photon과 같은 검증된 개발 도구 사용, 2인 협동 플레이로 멀티플레이 범위 고정, 기능 우선순위 기반의 단계적 개발 가능성을 근거로 판단하였다.

After 단계에서는 캐릭터 제어, 오브젝트 상호작용, 네트워크 동기화 구조가 실제 코드와 클래스 다이어그램으로 구체화되었다. 또한 Unity 실행 환경에서 온라인 장치 간 2인 접속, 스테이지 진행, 클리어 흐름까지 통합 테스트에 성공하여, 초기 계획에서 제시한 feasibility가 실제 구현 결과로 확인되었다.

3. Functional Requirements

Use Case: 플레이어, UI

담당: 배상혁

Use Case 1: 플레이어 이동 및 시점 조작

Use Case Name	플레이어 이동 및 시점 조작
Scenario	플레이어가 이동 입력과 마우스 시점 입력으로 캐릭터를 조작함
Triggering Event	로컬 입력 권한이 있는 플레이어가 Move, Look, Sprint, Jump 입력을 수행함
Brief Description	PlayerInputHandler가 입력을 PlayerNetworkInput으로 수집하고, PlayerMovement가 카메라 yaw 기준 이동, 회전, 점프, 스프린트 상태를 네트워크 틱에서 반영함
Actors	플레이어
Related Use Cases	Extend: ESC 설정 메뉴 열기
Stakeholders	플레이어, 게임 시스템
Preconditions	- 플레이어 오브젝트가 스폰되어 입력 권한을 보유해야 함 - PlayerInput, NetworkCharacterController, 카메라가 플레이어 프리팹에 연결되어 있어야 함
Post Conditions	- 플레이어 위치, 회전, 이동 입력, 점프/낙하/스프린트 상태가 갱신됨 - PlayerAnimatorController가 이동 및 공중 상태 애니메이션 파라미터를 반영함

Actor (플레이어)	System (게임 시스템)	Exception Conditions
1. 이동 및 시점 입력: WASD/스틱으로 이동 방향을 입력하고, 마우스/스틱으로 카메라 yaw와 pitch를 조작함. 2. 달리기와 점프 입력: 전방 이동 중 Sprint를 누르면 스프린트 상태가 되고, 지면 위에서 Jump를 누르면 점프가 실행됨.	3. 입력 수집 및 네트워크 전달: 로컬 입력 권한이 있는 클라이언트만 입력을 읽고, 이동 벡터와 카메라 각도 및 버튼 입력을 PlayerNetworkInput에 담음. 4. 이동 적용 및 애니메이션 반영: 네트워크 틱에서 카메라 방향 기준 이동, 회전, 점프 속도를 적용하고 이동/낙하/스프린트 애니메이션 값을 갱신함.	1. 입력 차단 상태: IsGameplayInputBlocked가 참이면 이동 벡터와 버튼 입력을 비우고 카메라 각도만 유지함. 2. 무입력 상태: 이동 입력이 없으면 MoveInput은 0으로 유지되고 캐릭터는 이동하지 않음. 3. 충돌 상태: 벽, 오브젝트, 무거운 퍼즐 오브젝트와 충돌하면 CharacterController와 push probe 정책에 따라 이동 성분이 제한됨.

Use Case 2: 물리 오브젝트 잡기/놓기/던지기

Use Case Name	물리 오브젝트 잡기/놓기/던지기
Scenario	플레이어가 카메라가 바라보는 물리 오브젝트를 들고 이동하거나 놓고, 조준 방향으로 던짐
Triggering Event	Grab, Throw 입력 또는 Grab 재입력으로 놓기 동작을 수행함
Brief Description	PlayerGrabHandler가 카메라 raycast로 GrabbableObject를 찾고, StateAuthority 또는 RPC를 통해 잡기/놓기/던지기를 요청함
Actors	플레이어
Related Use Cases	Include: 플레이어 이동 및 시점 조작
Stakeholders	플레이어, 게임 시스템
Preconditions	• 대상이 GrabbableObject이며 카메라 raycast와 사거리 검증을 통과해야 함 • 플레이어가 이미 다른 오브젝트를 들고 있지 않아야 함
Post Conditions	• 잡은 오브젝트는 플레이어의 hold point를 따라 이동함 • 놓기 또는 던지기 후 오브젝트의 물리 상태와 충돌 상태가 복구됨

Actor (플레이어)	System (게임 시스템)	Exception Conditions
1. 대상 조준 및 잡기: 카메라 중앙의 대상 오브젝트를 바라본 상태에서 Grab 입력을 눌러 오브젝트를 잡음. 3. 놓기 또는 던지기: 다시 Grab을 누르면 오브젝트를 놓고, Throw를 누르면 현재 조준 방향으로 오브젝트를 던짐.	2. 대상 검증 및 보유 상태 설정: 자기 콜라이더를 제외한 raycast hit를 거리순으로 검사하고, grabRange + serverValidationPadding 안의 대상만 잡기 처리함. 4. 물리 상태 전환: 들고 있는 동안 rigidbody, gravity, collision을 비활성화하고 hold point에 위치시키며, 해제 시 release overlap 검사를 통과한 위치에서 물리를 복구함.	1. 대상 없음: raycast가 잡을 수 없는 물체나 자기 콜라이더에만 닿으면 잡기 요청은 무시됨. 2. 사거리 초과: 대상의 collider closest point가 허용 사거리 밖이면 잡기 또는 상호작용이 실행되지 않음. 3. 해제 위치 충돌: 놓기/던지기 위치가 player body mask와 겹치면 해당 해제 동작은 적용되지 않음.

Use Case 3: 무거운 오브젝트 협동 밀기

Use Case Name	무거운 오브젝트 협동 밀기
Scenario	두 플레이어가 같은 방향으로 무거운 퍼즐 오브젝트를 밀어 제한 범위 안에서 이동시킴
Triggering Event	플레이어가 이동 중 무거운 오브젝트 방향으로 접촉하거나 push probe 범위 안에서 이동 입력을 유지함
Brief Description	PlayerMovement 가 이동 방향의 IPushable 대상에 push force 를 전달하고, HeavyObject 는 플레이어별 push sample 을 기록해 requiredPushers 이상이 같은 방향일 때 scripted movement 를 수행함
Actors	플레이어
Related Use Cases	Include: 플레이어 이동 및 시점 조작
Stakeholders	플레이어, 게임 시스템
Preconditions	- 대상 오브젝트가 HeavyObject이며 heavy puzzle layer로 설정되어 있어야 함 - 필요한 플레이어 수가 같은 방향으로 짧은 시간 안에 밀어야 함
Post Conditions	- 조건 충족 시 무거운 오브젝트가 방향별로 이동함 - 설정된 이동 범위가 있으면 x/z 좌표가 해당 범위 안으로 제한됨

Actor (플레이어)	System (게임 시스템)	Exception Conditions
1. 오브젝트 방향으로 이동: 플레이어가 무거운 오브젝트를 향해 이동하여 push probe 또는 CharacterController 충돌 판정을 발생시킴. 2. 협동 방향 유지: 두 플레이어가 같은 방향으로 계속 밀어 requiredPushers 조건을 만족시킴.	3. push sample 기록: StateAuthority가 플레이어별 push 방향과 시간을 기록하고, 최근 입력 중 같은 방향의 개수를 계산함. 4. scripted movement 적용: 조건이 충족되면 일정 시간 동안 오브젝트를 해당 방향으로 이동시키고, 설정된 min/max corner 범위로 위치를 제한함.	1. 인원 부족: 같은 방향의 active push 수가 requiredPushers보다 적으면 오브젝트는 이동하지 않음. 2. 방향 충돌: 같은 수의 서로 다른 방향 입력이 동시에 감지되면 이동 방향을 확정하지 않음. 3. 범위 설정 누락: 이동 범위 corner 중 하나만 지정된 경우 경고를 출력하고 scripted movement를 적용하지 않음.

Use Case 4: ESC 설정 메뉴 열기

Use Case Name	ESC 설정 메뉴 열기
Scenario	플레이어가 스테이지 진행 중 ESC 키로 설정 메뉴를 열고 닫거나 게임 나가기를 선택함
Triggering Event	MainStageScene에서 플레이어가 ESC 키를 누름
Brief Description	StageSettingsMenu가 ESC 입력을 감지해 설정 오버레이를 토글하고, 메뉴가 열려 있는 동안 게임플레이 입력과 크로스헤어를 차단한 뒤 커서를 UI 조작 상태로 전환함
Actors	플레이어
Related Use Cases	Include: 플레이어 이동 및 시점 조작
Stakeholders	플레이어, 게임 시스템
Preconditions	- 현재 씬이 MainStageScene이어야 함 - StageSettingsMenu가 씬 로드 시 생성되고 StageSettingsCanvas가 준비되어 있어야 함
Post Conditions	- 메뉴가 열리면 Settings 오버레이와 Close/Leave Game 버튼이 표시됨 - 메뉴가 닫히면 게임플레이 입력, 커서 상태, 크로스헤어 표시 상태가 이전 상태로 복구됨

Actor (플레이어)	System (게임 시스템)	Exception Conditions
1. 메뉴 열기: 스테이지 진행 중 ESC 키를 눌러 Settings 오버레이를 호출함. 3. 메뉴 닫기 또는 나가기: Close 버튼이나 ESC 키로 메뉴를 닫거나, Leave Game 버튼으로 세션을 떠나 로비로 이동함.	2. 입력 차단 및 UI 표시: 메뉴를 열 때 PlayerInputHandler.IsGameplayInput Blocked를 true로 설정하고 로컬 PlayerInput과 크로스헤어를 비활성화한 뒤 커서를 표시함. 4. 상태 복구 또는 씬 전환: Close 시 이전 입력/커서/크로스헤어 상태를 복구하고, Leave Game 시 세션을 떠난 뒤 로비 씬으로 이동함.	1. 메인 스테이지 아님: 현재 씬이 MainStageScene이 아니면 StageSettingsMenu 오브젝트를 생성하지 않거나 즉시 제거함. 2. 나가기 처리 중: isLeaving 상태에서는 ESC 입력과 Close 처리를 무시하고 중복 나가기 요청을 막음. 3. 매니저 누락: SceneFlowManager가 없으면 로비 씬 직접 로드 또는 빌드 인덱스 0 로드로 fallback 처리함.

Use Case: 서버

담당: 이규원

Use Case 1: 방 생성하기

Use Case Name	방 생성하기 (Create Room)
Scenario	사용자가 게임서버에 접속하여 새로운 방을 생성함
Triggering Event	사용자가 '방 생성하기' 버튼을 누름
Brief Description	시스템은 플레이어의 요청을 받아 새로운 Photon Room을 생성하며, 이때 방 이름, 최대 인원, 공개 여부 등의 속성을 설정함
Actors	플레이어
Related Use Cases	Include: 방 참가하기 Extend: 게임 시작하기
Stakeholders	플레이어, 같은 방에 입장할 다른 플레이어들, 게임 시스템
Preconditions	- 플레이어가 서버에 연결되어 있어야 함 - 방 생성 화면 또는 로비 상태에 있어야 함 - 방 생성에 필요한 정보(방 이름, 최대 인원 등)가 입력되어야 함
Post Conditions	- 새로운 방이 생성되고, 방을 생성한 플레이어가 즉시 입장됨 - 방 정보(속성)가 설정됨 - 공개 방이면 타인의 검색/입장이 가능하며, 비공개 방이면 이름을 아는 사용자만 입장 가능함

Actor (플레이어)	System (Photon 서버)	Exception Conditions
1. 메뉴 선택 및 정보 입력: 방 생성 메뉴를 선택한 뒤, 방 이름, 최대 인원 등의 속성 정보를 입력함. 2. 생성 요청: 생성 버튼을 눌러 서버에 방 생성 요청을 전송함.	3. 유효성 검사 및 방 생성: 입력값의 유효성을 확인한 후, 새로운 방을 생성하고 방 정보를 서버에 저장함. 4. 플레이어 입장 및 UI 갱신: 방 생성이 완료되면 플레이어를 해당 방에 입장시키고 대기실 화면을 표시함.	1. 연결 오류: 서버 연결이 끊어진 상태일 경우 방 생성 실패 처리 및 알림. 2. 입력값 오류: 중복되거나 유효하지 않은 방 이름, 또는 입력값이 누락된 경우 생성 거부. 3. 서버 요청 실패: 네트워크 문제 등으로 Photon 서버 생성 요청에 실패한 경우 에러 메시지 출력. 4. 잘못된 속성: 허용되지 않은 방 속성(예: 비정상적인 최대 인원 수) 설정 시 생성 제한.

Use Case 2: 방 참가하기

Use Case Name	방 참가하기 (Join Room)
Scenario	플레이어가 기존에 생성된 방에 입장하기 위해 참가 요청을 보냄
Triggering Event	플레이어가 방 목록에서 특정 방을 선택하거나, 방 이름을 입력하여 참가를 요청함
Brief Description	시스템은 플레이어의 방 참가 요청을 받아 해당 방의 존재 여부, 입장 가능 여부, 최대 인원 등을 기준으로 참가를 처리함
Actors	플레이어
Related Use Cases	Include: 서버 접속하기, 방 검색하기 Extend: 방 생성하기
Stakeholders	플레이어, 같은 방의 기존 플레이어들, 게임 시스템
Preconditions	• 플레이어가 Photon 서버에 연결되어 있어야 함 • 참가하려는 방이 존재하거나 조건에 맞는 방이 있어야 함 • 방이 닫혀 있지 않고 최대 인원에 도달하지 않아야 함
Post Conditions	• 플레이어가 해당 방에 입장하며 방 인원 수가 갱신됨 • 같은 방의 기존 플레이어들에게 새 참가자의 입장이 동기화됨

Actor (플레이어)	System (Photon 서버)	Exception Conditions
1. 방 탐색 및 선택: 방 목록을 확인하고 참가할 방을 선택하거나 방 이름을 직접 입력함. 2. 참가 요청: 참가 버튼을 눌러 서버에 해당 방으로의 입장을 요청함.	3. 방 상태 검증: 요청받은 방의 존재 여부, 현재 인원 수 및 입장 가능 상태(비밀번호 등)를 확인함. 4. 입장 처리 및 갱신: 검증 통과 시 플레이어를 방에 입장시키고, 참가자 목록을 갱신하여 대기실 화면을 표시함.	1. 방 존재 안 함: 방이 이미 사라졌거나 존재하지 않는 경우 참가 실패 처리 및 알림. 2. 인원 초과: 방이 이미 최대 인원에 도달하여 가득 찬 경우 입장 거부 및 에러 메시지 출력. 3. 중복 입장: 동일한 UserID로 같은 방에 이미 입장해 있는 상태에서 중복 참가를 요청할 경우 처리 거부.

Use Case 3: 채팅하기

Use Case Name	채팅하기 (Chatting)
Scenario	같은 게임 방에 입장한 플레이어들이 대기실 또는 플레이 중 텍스트 메시지를 주고받음
Triggering Event	플레이어가 채팅창에 메시지를 입력하고 전송(Enter) 키를 누름
Brief Description	시스템은 플레이어가 입력한 메시지를 RPC(Remote Procedure Call)를 통해 같은 방의 다른 플레이어들에게 전달하고 UI에 표시함
Actors	플레이어
Related Use Cases	Include: 서버 접속하기, 방 생성하기, 방 참가하기
Stakeholders	플레이어, 같은 방의 다른 플레이어, 게임 시스템
Preconditions	- 플레이어가 서버에 연결되어 있고, 같은 방에 입장한 상태여야 함 - 채팅 UI가 활성화되어 있어야 함 - 채팅 처리 오브젝트에 네트워크 컴포넌트(PhotonView)가 연결되어 있어야 함
Post Conditions	- 전송된 메시지가 방 안의 모든 플레이어에게 전달됨 - 발신자 정보와 함께 각 플레이어의 채팅창 화면에 메시지가 표시됨

Actor (플레이어)	System (Photon 서버)	Exception Conditions
1. 메시지 작성: 채팅 입력창을 활성화하고 전송할 텍스트 메시지를 작성함. 2. 전송 요청: Enter 키 또는 전송 버튼을 눌러 메시지 발송을 서버에 요청함.	3. 유효성 검증 및 브로드캐스트: 입력된 메시지가 비어 있는지 확인한 후, RPC를 통해 방 안의 다른 플레이어들에게 메시지를 전파함. 4. UI 갱신: 각 클라이언트의 채팅 UI에 발신자 정보와 함께 수신된 메시지를 출력함.	1. 연결 오류: 서버 연결이 끊어진 경우 메시지 전송이 실패하며 에러 처리됨. 2. 방 미입장 상태: 로비 등 방에 입장하지 않은 상태에서는 채팅 기능을 사용할 수 없음. 3. 빈 메시지 전송: 입력된 텍스트가 비어 있거나 공백만 있는 경우, 시스템이 전송 요청을 무시함.

Use Case 4: 스테이지 시작하기

Use Case Name	스테이지 시작하기 (Start Stage)
Scenario	같은 방에 있는 플레이어들의 준비 상태가 충족되면 시스템이 게임 시작 여부를 방 전체에 반영함
Triggering Event	게임 시작 조건이 충족되거나, 플레이어(방장)가 시작 요청을 보냄
Brief Description	시스템은 게임 시작 여부, 시작 시간 등의 정보를 룸 속성(Room Properties)으로 갱신하고, 같은 방의 모든 플레이어에게 이를 동기화함
Actors	플레이어
Related Use Cases	Include: 방 참가하기, Ready 하기, 방 인원 Ready 체크
Stakeholders	플레이어, 같은 방의 다른 플레이어, 게임 시스템
Preconditions	- 플레이어들이 같은 Photon Room에 입장해 있어야 함 - 모든 플레이어의 게임 시작 조건(Ready 등)이 충족되어야 함 - 시작 상태를 저장할 룸 속성(HashTable 형태)이 정의되어 있어야 함
Post Conditions	- 게임 시작 여부가 룸 전체 상태 속성에 갱신/반영됨 - 필요 시 시작 시간 및 라운드 정보가 함께 저장됨 - 같은 방의 모든 플레이어 화면이 동일한 인게임 상태로 전환됨

Actor (플레이어)	System (Photon 서버)	Exception Conditions
1. 대기 및 준비: 같은 방에 입장하여 게임 시작을 위한 준비(Ready) 상태로 대기함. 2. 시작 요청: 조건이 충족된 상태에서 플레이어(방장)가 게임 시작 버튼을 누름.	3. 시작 조건 검증: 시작 요청을 수신한 뒤, 현재 방 인원과 모든 플레이어의 준비 상태를 확인함. 4. 룸 속성 갱신 및 전환: 시작이 가능하다고 판단되면 룸 속성(시작 시간, 상태)을 갱신하고 모든 플레이어의 화면을 게임 진행 상태로 일제히 전환함.	1. 인원 부족: 일부 플레이어가 아직 방에 입장하지 않았거나 최소 시작 인원에 미달한 경우. 2. 준비 미완료: 방 안의 모든 플레이어가 'Ready' 상태가 아닌 경우 시작 요청 거절. 3. 속성 갱신 실패: 네트워크 문제 등으로 Photon 룸 속성(HashTable) 갱신에 실패한 경우 진행 중단.

Use Case: 오브젝트, 맵

담당: 양준영

Use Case 1: 스테이지 입장 - 맵 생성

Use Case Name	스테이지 입장 - 맵 생성 (Load Stage Map)
Scenario	플레이어가 스테이지를 선택하면 맵이 생성되고 아바타가 배치됨
Triggering Event	플레이어가 스테이지를 선택하여 입장을 시도함
Brief Description	게임 시스템은 플레이어가 선택한 스테이지를 확인하여 해당하는 맵을 로딩하고, 완료 시 플레이어의 아바타를 스폰 위치에 생성함
Actors	플레이어
Related Use Cases	Include: 서버 - 게임 시작하기
Stakeholders	플레이어, 게임 시스템
Preconditions	• 게임이 실행 중이어야 함 • 플레이어들이 게임에 접속해 시작할 준비를 마친 상태여야 함
Post Conditions	• 스테이지의 맵이 로드됨 • 게임에 접속한 플레이어들의 아바타가 맵의 지정된 위치에 놓임

Actor (플레이어)	System (게임 시스템)	Exception Conditions
1. 스테이지 선택: 스테이지 목록이 나열된 화면에서 플레이할 스테이지를 찾아 선택함.	2. 씬 이동 및 맵 로딩: 맵을 로딩할 씬으로 이동한 후, 선택된 스테이지를 확인하여 해당하는 맵 데이터를 매니저를 통해 로딩함. 3. 아바타 스폰 및 준비 완료: 로딩이 완료되면 플레이어의 아바타를 생성해 맵의 스폰 포인트에 위치시키고, 게임을 시작할 수 있도록 처리함.	1. 연결 끊김: 로딩이 진행되는 과정에서 플레이어와의 네트워크 연결이 끊긴다면, 알림 팝업을 띄우고 즉시 로비로 이동 처리함.

Use Case 2: 기믹 수행 - 레버 작동

Use Case Name	기믹 수행 - 레버 작동 (Operate Lever Gimmick)
Scenario	플레이어가 레버 오브젝트와 상호작용해 숨겨진 구조물을 활성화함
Triggering Event	플레이어가 레버 오브젝트와 상호작용을 시도함
Brief Description	플레이어가 레버를 작동시키면, 아바타 조작만으로는 오를 수 없는 절벽에 비계 형태의 구조물이 나타나 이동 가능한 길을 만들어 줌
Actors	플레이어
Related Use Cases	없음
Stakeholders	플레이어, 게임 시스템
Preconditions	• 게임이 실행 중이며 플레이어가 아바타를 조작할 수 있는 상태여야 함 • 오를 수 없는 절벽 지형과 상호작용 가능한 레버가 존재해야 함
Post Conditions	• 레버가 작동 후 상호작용 불가(비활성화) 상태로 전환됨 • 절벽을 오를 수 있는 비계 형태의 구조물이 맵에 생성/등장함

Actor (플레이어)	System (게임 시스템)	Exception Conditions
1. 상호작용 시도: 절벽 근처에 위치한 레버 오브젝트에 다가가 상호작용 키를 입력함.	2. 상태 비활성화: 레버와의 상호작용 입력을 감지하면 즉시 해당 레버를 상호작용 불가 상태로 변경함. 3. 구조물 활성화: 절벽 속이나 땅 아래에 숨어 있던 비계 형태의 지형 구조물을 맵 상에 등장시킴.	1. 중복 작동 시도: 이미 작동이 완료되어 비활성화된 레버에 다시 상호작용을 시도할 경우, 시스템은 이를 무시하고 아무런 이벤트도 발생시키지 않음.

Use Case 3: 기믹 수행 - 발판 작동

Use Case Name	기믹 수행 - 발판 작동 (Operate PressurePlate Gimmick)
Scenario	플레이어가 발판 오브젝트와 상호작용해서 구조물이 움직이게 함
Triggering Event	두 플레이어가 각각 발판 오브젝트 위에 올라섬
Brief Description	플레이어들이 발판 오브젝트 위에 올라감. 발판이 작동하면 플랫폼 구조물이 움직이기 시작. 발판 위에서 내려오면, 움직이던 플랫폼 구조물은 움직임을 멈춤
Actors	플레이어
Related Use Cases	없음
Stakeholders	플레이어, 게임 시스템
Preconditions	• 게임이 실행 중이며 플레이어가 아바타를 조작할 수 있는 상태여야 함 • 두 개의 발판과 플랫폼 오브젝트가 존재해야 함
Post Conditions	플랫폼 구조물이 지속적으로 움직이기 시작함

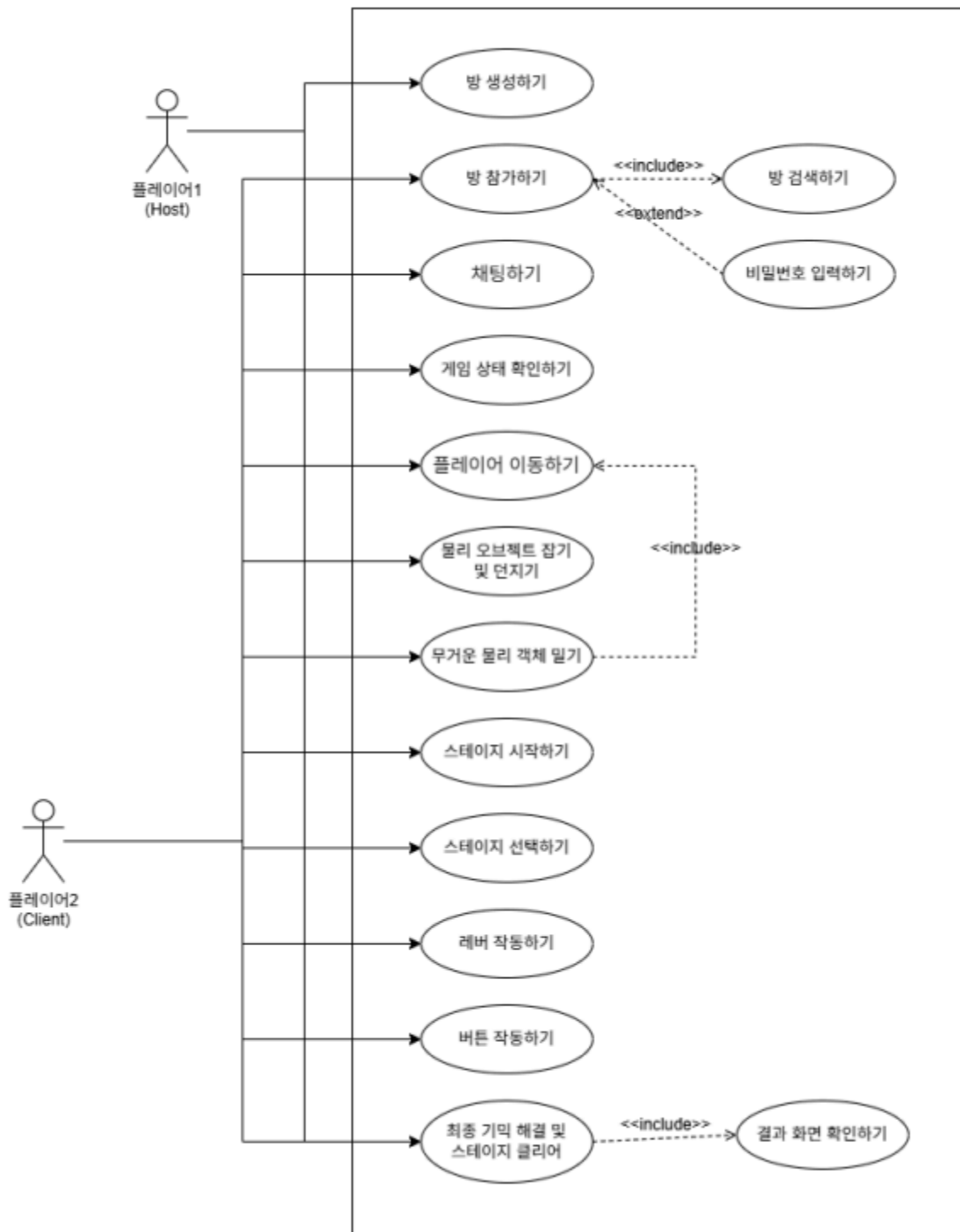
Actor (플레이어)	System (게임 시스템)	Exception Conditions
1. 최초 상호작용 시도: 두 명의 플레이어가 두 개의 발판 오브젝트 위에 각각 올라감 3. 추가 상호작용 시도: 두 플레이어 중 최소 한 명이 발판 위에서 내려감	2. 1차 상태 전환: 플랫폼 오브젝트가 지속적으로 움직이기 시작함 4. 2차 상태 전환: 플랫폼 오브젝트의 움직임을 멈춤	1. 발판에 정확히 위치 : 발판 오브젝트의 트리거 구역에 정확하게 위치하지 않으면 발판은 반응하지 않음.

Use Case 4: 최종 기믹 해결 및 스테이지 클리어

Use Case Name	최종 기믹 해결 및 스테이지 클리어 (Clear Final Gimmick & Stage)
Scenario	스테이지의 최종 기믹을 수행하고 스테이지 클리어 상태로 진입함
Triggering Event	플레이어들이 스테이지에 설정된 최종 클리어 조건을 달성함
Brief Description	두 명의 플레이어가 각각 두 개의 발판을 밟아 출구를 개방하고, 두 플레이어 모두 해당 출구로 진입하여 스테이지를 클리어함
Actors	Primary: 플레이어 A, 플레이어 B
Related Use Cases	Include: 플레이어 - 스테이지 이동 및 클리어하기
Stakeholders	플레이어, 게임 시스템
Preconditions	- 게임이 실행 중이며 아바타를 조작할 수 있는 상태여야 함 - 클리어 조건에 해당하는 두 개의 발판과 닫혀있는 출구가 존재해야 함
Post Conditions	시스템이 스테이지가 클리어된 것을 확인하고 클리어 관련 절차에 돌입함

Actor (플레이어 A, B)	System (게임 시스템)	Exception Conditions
1. 발판 활성화 (기믹 수행): 두 명의 플레이어가 각각 맵에 배치된 두 개의 발판 오브젝트 위에 올라섬. 3. 출구 진입: 기믹이 풀려 출구 오브젝트가 열리면, 두 플레이어 모두 해당 출구 영역으로 진입함.	2. 기믹 판정 및 출구 개방: 두 개의 발판이 모두 밟혀 활성화된 것을 감지하면, 닫혀있던 출구 오브젝트를 활성화하여 문을 에. 4. 클리어 판정 및 절차 실행: 두 플레이어의 출구 진입이 모두 감지되면 스테이지 클리어 조건을 충족시키고 클리어 절차를 실행함.	1. 연결 끊김: 스테이지 클리어 관련 절차를 실행하기 이전에 네트워크 연결이 끊긴다면, 실행 절차를 즉시 중단하고 재시도 팝업을 띄우거나 메인 메뉴로 강제 이동시킴.

3. Use Case Diagram



4. NFR

Player

Use Case Name	NFR 내역	Quality	Quality Attributes
플레이어 이동 및 시점 조작	입력 응답성: 로컬 입력 권한을 가진 플레이어만 입력을 수집하고, 입력은 다음 네트워크 틱에 반영되어야 함.	Performance Efficiency (성능 효율성)	OnInput에서 입력을 설정하고, 버튼 입력은 전송 후 즉시 버퍼를 비움.
플레이어 이동 및 시점 조작	이동 상태 일관성: 이동 벡터, 카메라 각도, 스프린트, 점프, 낙하 상태가 일관되게 반영되어야 함.	Functional Suitability (기능 적합성)	PlayerMovement의 Networked 값과 애니메이션 파라미터를 같은 상태 기준으로 갱신함.
물리 오브젝트 잡기/놓기/던지기	상호작용 정확성: 대상은 카메라 raycast, 자기 collider 제외, 사거리 검증을 모두 통과해야 함.	Functional Suitability (기능 적합성)	grabRange + serverValidationPadding 안의 유효 대상만 RPC/StateAuthority로 처리함.
물리 오브젝트 잡기/놓기/던지기	물리 상태 안정성: 들고 있는 동안 충돌/중력을 비활성화하고, 해제 시 겹침 검사를 통과해야 함.	Reliability (신뢰성)	held 모드로 물리를 전환하고, release overlap 실패 시 drop/throw를 거부함.
무거운 오브젝트 협동 밀기	협동 판정 정확성: 요구 인원 이상이 같은 방향으로 최근 push sample을 남긴 경우에만 이동해야 함.	Functional Suitability (기능 적합성)	requiredPushers 이상이고 방향 동물이 없을 때만 scripted movement를 활성화함.
무거운 오브젝트 협동 밀기	이동 범위 안정성: scripted movement 중 오브젝트가 지정된 x/z 범위를 벗어나지 않아야 함.	Reliability (신뢰성)	min/max corner가 있으면 좌표를 clamp하고, 범위 설정이 불완전하면 이동하지 않음.
ESC 설정 메뉴 열기	메뉴 조작 사용성: ESC 키로 설정 메뉴를 열고 닫으며, 닫기와 게임 나가기 선택지를 제공해야 함.	Usability (사용성)	StageSettingsMenu가 오버레이와 버튼을 생성하고 ESC로 open/close를 토글함.
ESC 설정 메뉴 열기	입력 차단 일관성: 메뉴가 열린 동안 조작 입력과 크로스헤어를 차단하고, 닫으면 이전 상태로 복구해야 함.	Functional Suitability (기능 적합성)	IsGameplayInputBlocked, PlayerInput, showCrosshair, Cursor 상태를 저장/복구함.

Server

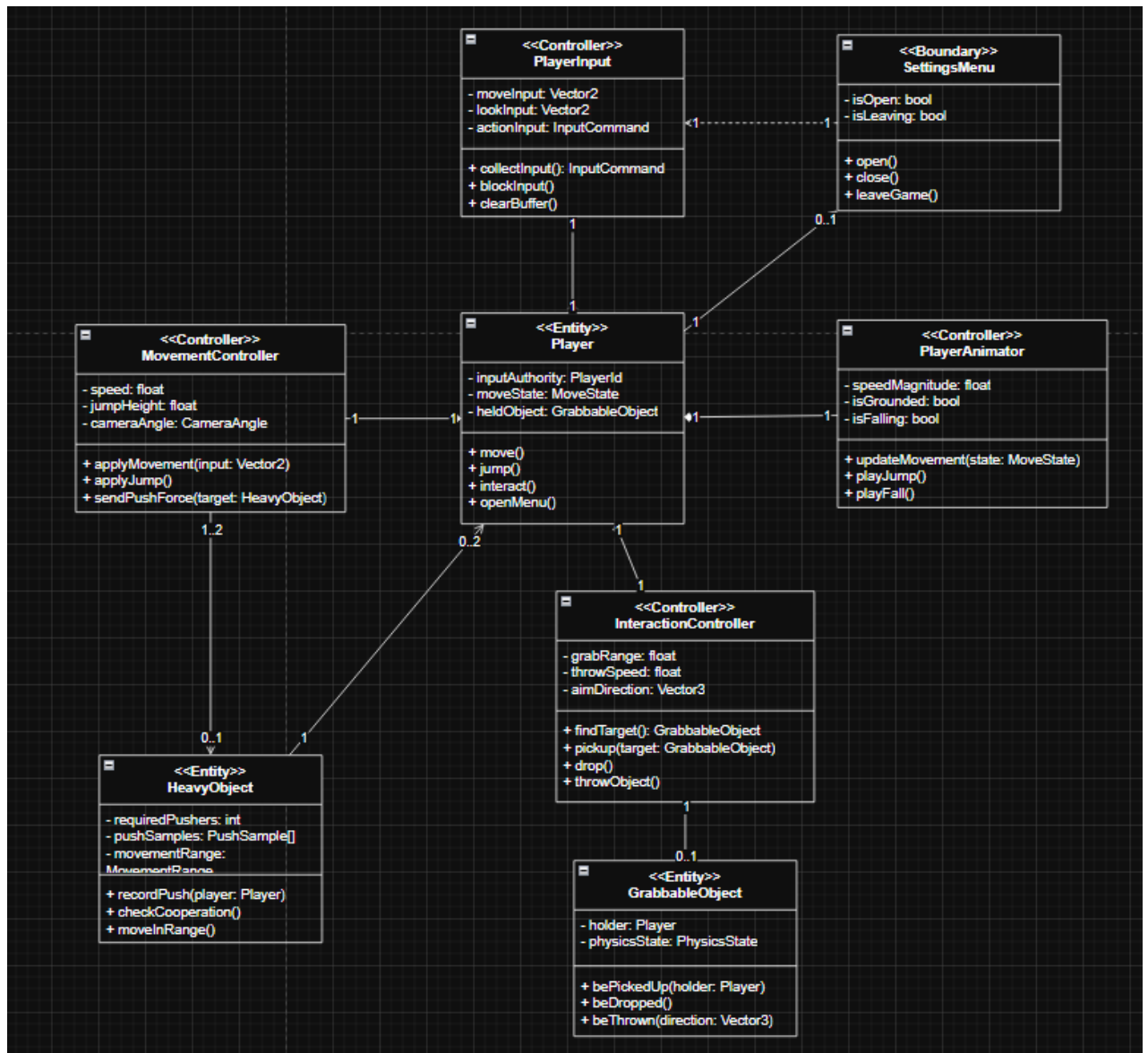
Use Case Name	NFR 내역	Quality	Quality Attributes
방 생성하기	방 생성 응답성: 사용자가 방 생성 요청을 보냈을 때, 시스템은 지연 없이 방 생성 결과를 반환하고 대기실 화면으로 전환해야 함.	Performance Efficiency (성능 효율성)	Time Behavior(시간 반응성) 평균 7초 이내에 방 생성 결과가 화면에 반영되고 대기실로 전환되어야 함.
방 참가하기	입장 판정 정확성: 존재하지 않는 방, 최대 인원을 초과한 방, 닫힌 방에 대해서는 참가 요청이 정확하게 거부되어야 함.	Functional Suitability (기능 적합성)	Fault Tolerance(결함 수용성) 동시 요청 상황에서도 최대 인원을 초과한 입장을 허용하지 않아야 함.
채팅하기	메시지 전달 응답성: 플레이어가 전송한 채팅 메시지는 같은 방의 다른 플레이어들에게 짧은 시간 안에 표시되어야 함.	Performance Efficiency (성능 효율성)	Confidentiality(기밀성) 같은 방 외부의 사용자에게 채팅 메시지가 전달되거나 노출되지 않아야 함.
스테이지 시작하기	시작 조건 판정 정확성: 모든 플레이어가 Ready 상태일 때만 게임 시작이 허용되어야 하며, 조건이 충족되지 않으면 시작 요청이 거부되어야 함.	Functional Suitability (기능 적합성)	Interoperability(상호운용성) 모든 클라이언트가 동일한 시작 상태와 시작 시점을 공유해야 함.

Object

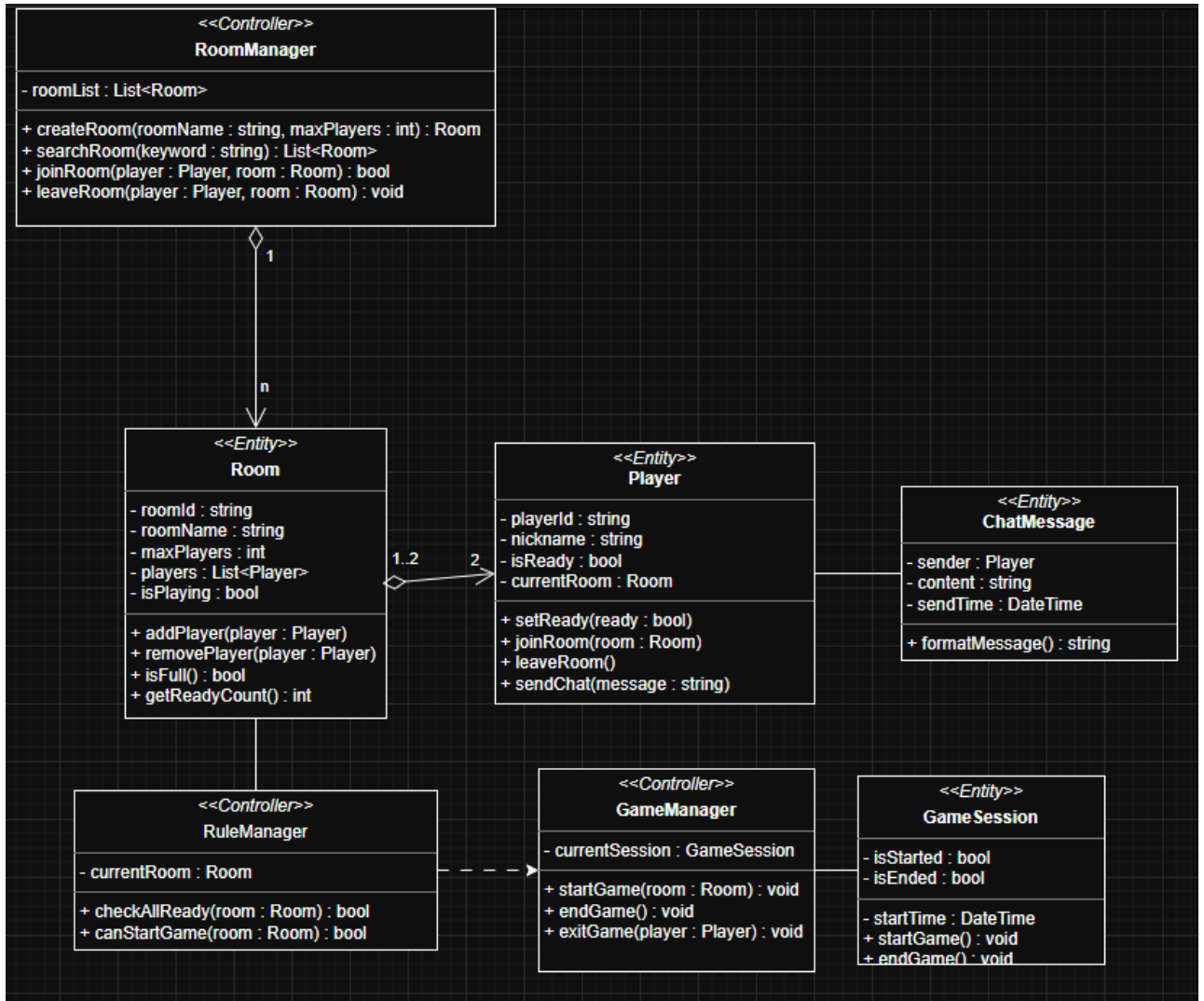
Use Case Name	NFR 내역	Quality	Quality Attributes
스테이지 입장 - 맵 생성	스테이지 생성 응답성: 플레이어가 스테이지를 선택하고 게임을 시작하려 할 때 맵 로딩이 신속히 진행되어야 함.	Performance Efficiency (성능 효율성)	Time Behavior: 지정된 시간(약 3초) 안에 스테이지가 로딩되도록 함.
기믹 수행 - 레버 작동	상호작용 정확성: 레버 상호작용 시 지정된 오브젝트들이 정확히 작동하도록 해야 함.	Functional Suitability (기능 적합성)	Functional Correctness: 레버와 지정된 오브젝트를 정확히 연결해 함수가 올바르게 작동하도록 구현.
기믹 수행 - 버튼 작동	기믹 학습성: 복잡한 기믹이라도 사용자가 빠른 시간 내에 이해할 수 있게 설계해야 함.	Interaction Capability (상호작용 능력)	Learnability: 한 화면 안에서 기믹의 실행이 파악되는 등 기믹을 직관적으로 이해 가능하게 설계.
스테이지 최종 기믹 해결 및 스테이지 클리어	클리어 판정 무결성: 최종 기믹을 정상적으로 수행했을 시에만 스테이지 클리어 판정으로 인식해야 함.	Reliability (안정성)	Faultlessness: 지정된 기믹만 클리어 판정으로 연결, 이외 모든 기믹에 대해 클리어 판정으로 연결되지 않도록 전수조사

5. Conceptual Class Diagram

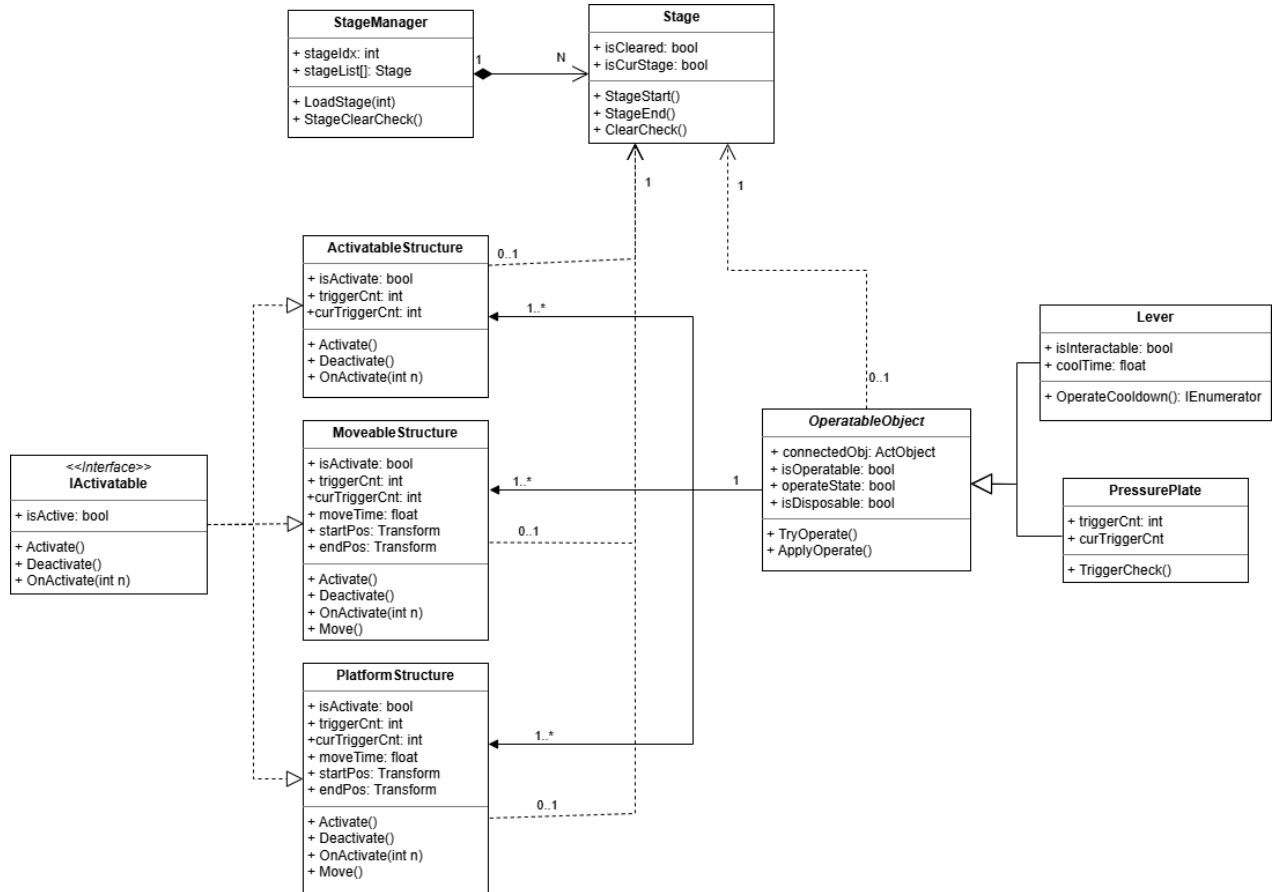
Player



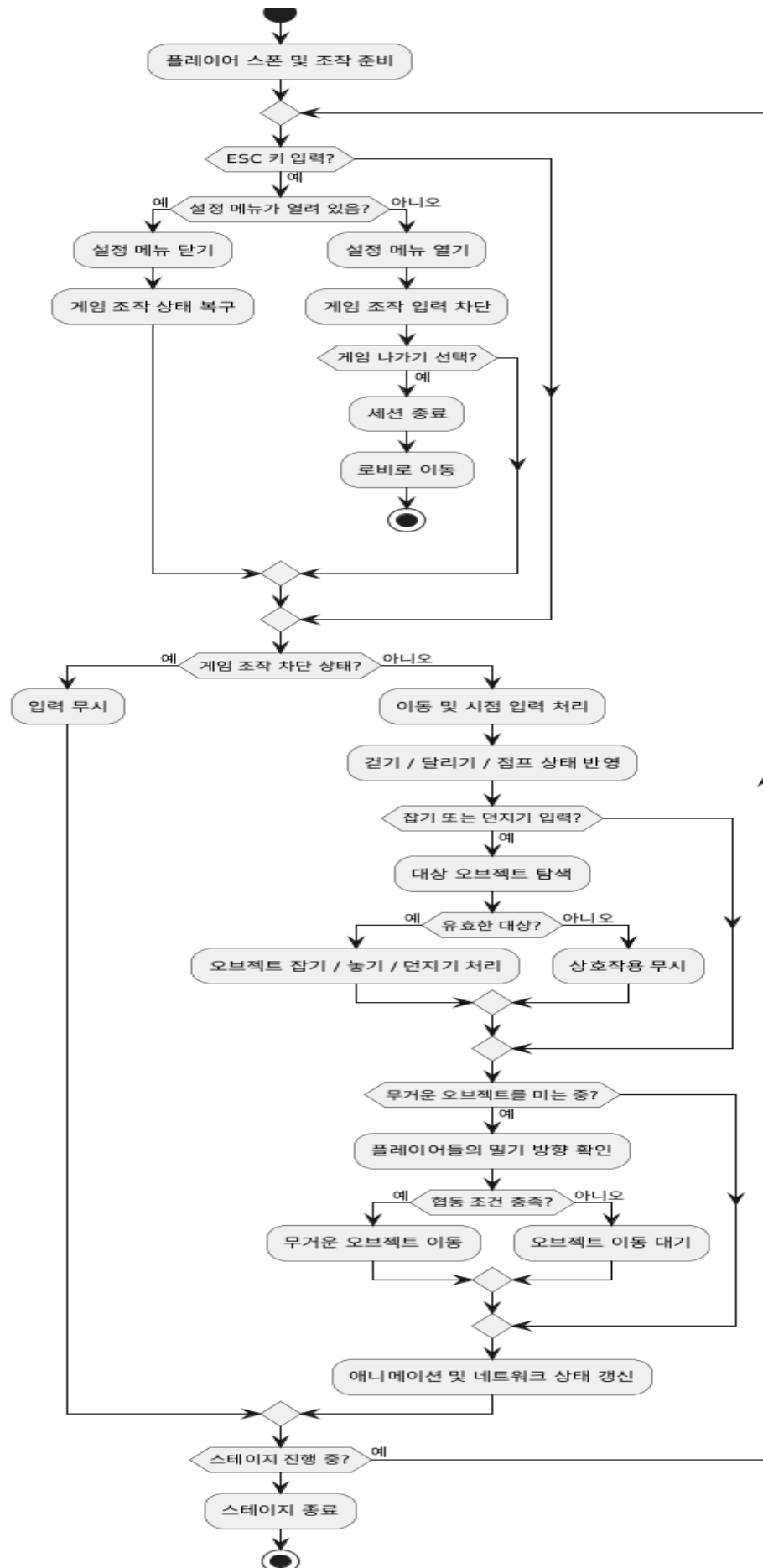
Server



Object



6. Activity Diagram Player

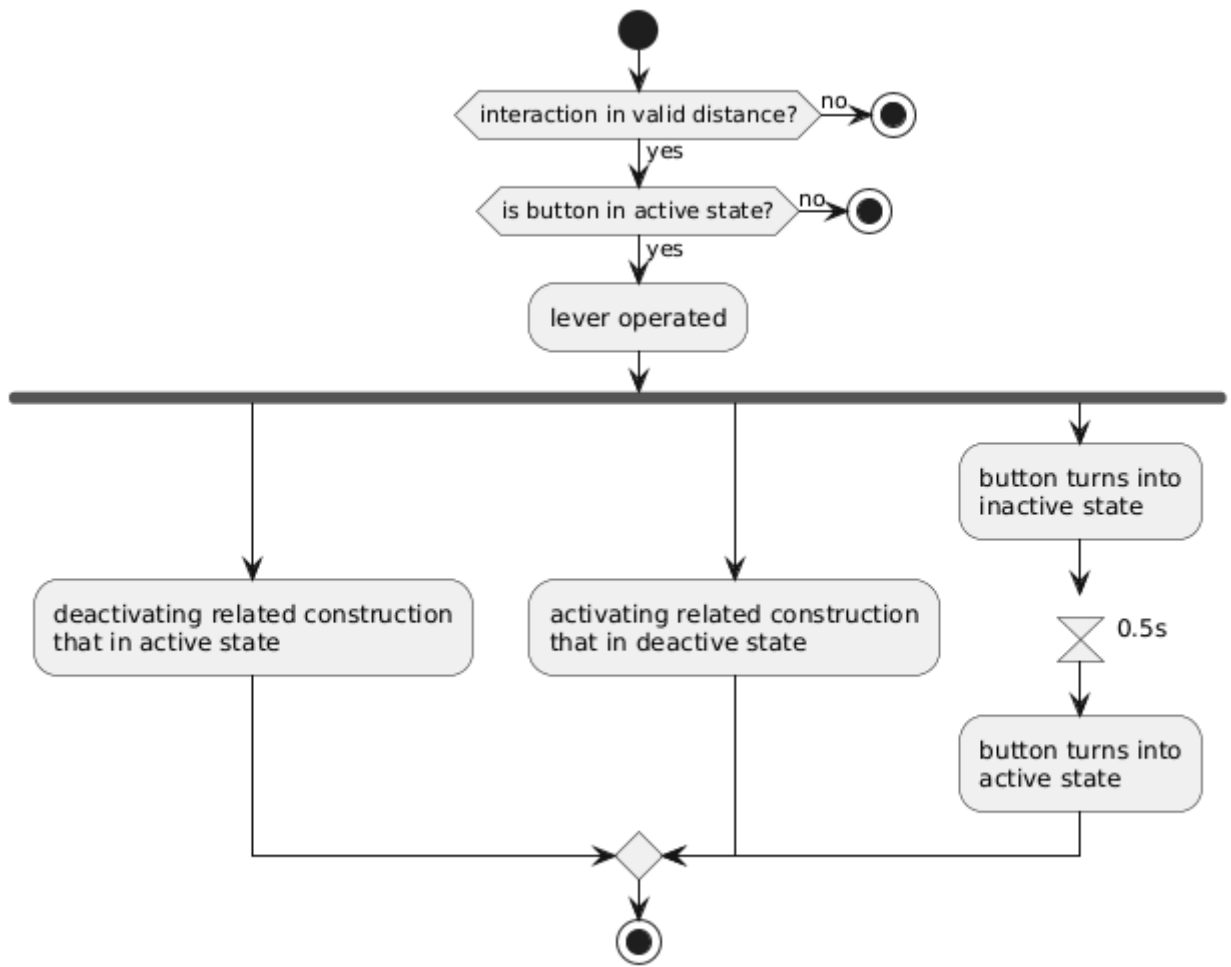


Server

방 입장(Room Join) 유즈케이스 Activity Diagram



Object



7-1. System Sequence Diagram

시스템 시퀀스 다이어그램



7-1. Operation Contract

System Operation	Responsibility	Pre-condition	Post-condition
<code>createRoom(roomName, maxPlayers, isPublic)</code>	플레이어 1이 새 방 생성을 요청하고 대기실 입장을 시작한다.	플레이어 1이 로비에 있으며 방 생성에 필요한 값을 입력했다.	방 생성 성공 시 <code>enterWaitingRoom(roomId)</code> 가 반환된다. 실패 시 <code>showCreateRoomError(reason)</code> 가 반환된다.
<code>joinRoom(roomId)</code>	플레이어 2가 기존 방 참가를 요청하고 대기실 입장을 시작한다.	플레이어 2가 로비에 있으며 참가할 <code>roomId</code> 를 입력했다.	참가 성공 시 <code>showWaitingRoom(roomInfo)</code> 가 반환된다. 실패 시 <code>joinRoomRejected(reason)</code> 가 반환된다.
<code>sendChatMessage(message)</code>	대기실에서 플레이어 메시지를 같은 방 참가자에게 전달한다.	플레이어가 방에 입장에 있고 채팅 입력이 가능하다.	메시지가 플레이어 1, 플레이어 2의 채팅 화면에 표시된다. 처리할 수 없으면 요청자에게 거절 결과가 반환된다.
<code>setReady(playerId, isReady)</code>	각 플레이어의 준비 상태를 대기실 상태에 반영한다.	플레이어가 대기실에 입장에 있다.	해당 플레이어의 준비 상태가 갱신되고 대기실 참가자에게 공유된다.
<code>requestStartStage()</code>	플레이어 1이 스테이지 시작을 요청한다.	플레이어 1이 방장 역할이며 스테이지를 시작할 수 있는 대기실 상태이다.	시작 가능하면 두 플레이어에게 <code>loadStage(stageId)</code> 가 전달된다. 시작할 수 없으면 플레이어 1에게 <code>startStageRejected(reason)</code> 가 반환된다.
<code>enterStage(stageId)</code>	플레이어가 스테이지 입장을 요청하고 게임 진행 상태로 전환한다.	스테이지 시작 요청이 수락되었고 <code>stageId</code> 가 정해져 있다.	입장 성공 시 <code>stageReady(stageId)</code> 가 반환된다. 실패 시 <code>returnToLobbyWithError(reason)</code> 가 반환된다.

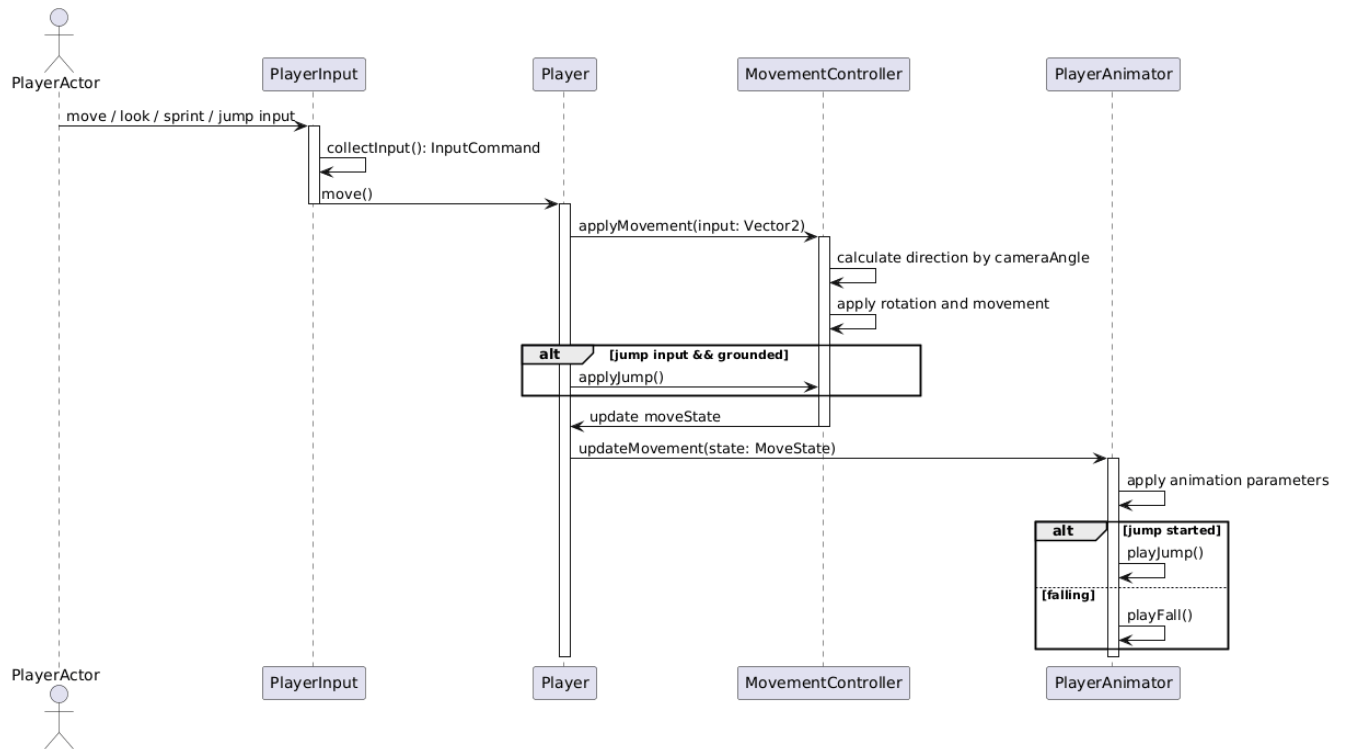
System Operation	Responsibility	Pre-condition	Post-condition
<code>inputMovement()</code>	플레이어 이동 입력을 스테이지 상태에 반영한다.	스테이지가 진행 중이고 플레이어 조작이 가능한 상태이다.	플레이어 위치와 화면에 반영될 상태가 <code>updateStageState(result)</code> 로 갱신된다.
<code>requestGrab()</code> <code>operateLever()</code> <code>toggleStageSettingsMenu()</code>	플레이어의 오브젝트 잡기, 레버 조작, 설정 메뉴 요청을 처리한다.	플레이어가 스테이지 안에 있으며 대상 오브젝트나 UI를 조작할 수 있다.	오브젝트, 구조물, 메뉴 상태가 변경되고 결과가 <code>updateStageState(result)</code> 로 반환된다.
<code>pushObject()</code> <code>operateButton()</code> <code>stepOnPressurePlate()</code>	플레이어의 협동 밀기, 버튼 조작, 발판 활성화 요청을 처리한다.	플레이어가 스테이지 안에 있으며 협동 기믹 수행 위치에 있다.	협동 오브젝트, 다리, 최종 발판 상태가 변경되고 결과가 <code>updateStageState(result)</code> 로 반환된다.
<code>enterExitArea()</code>	플레이어의 출구 진입을 기록하고 스테이지 클리어 결과를 반환한다.	출구가 열려 있으며 플레이어가 출구 영역에 진입했다.	두 플레이어가 모두 진입하면 <code>stageCleared()</code> 가 반환된다. 일부만 진입한 경우 <code>waitForPartner()</code> 가 반환된다.

7-2. Sequence Diagram

Player

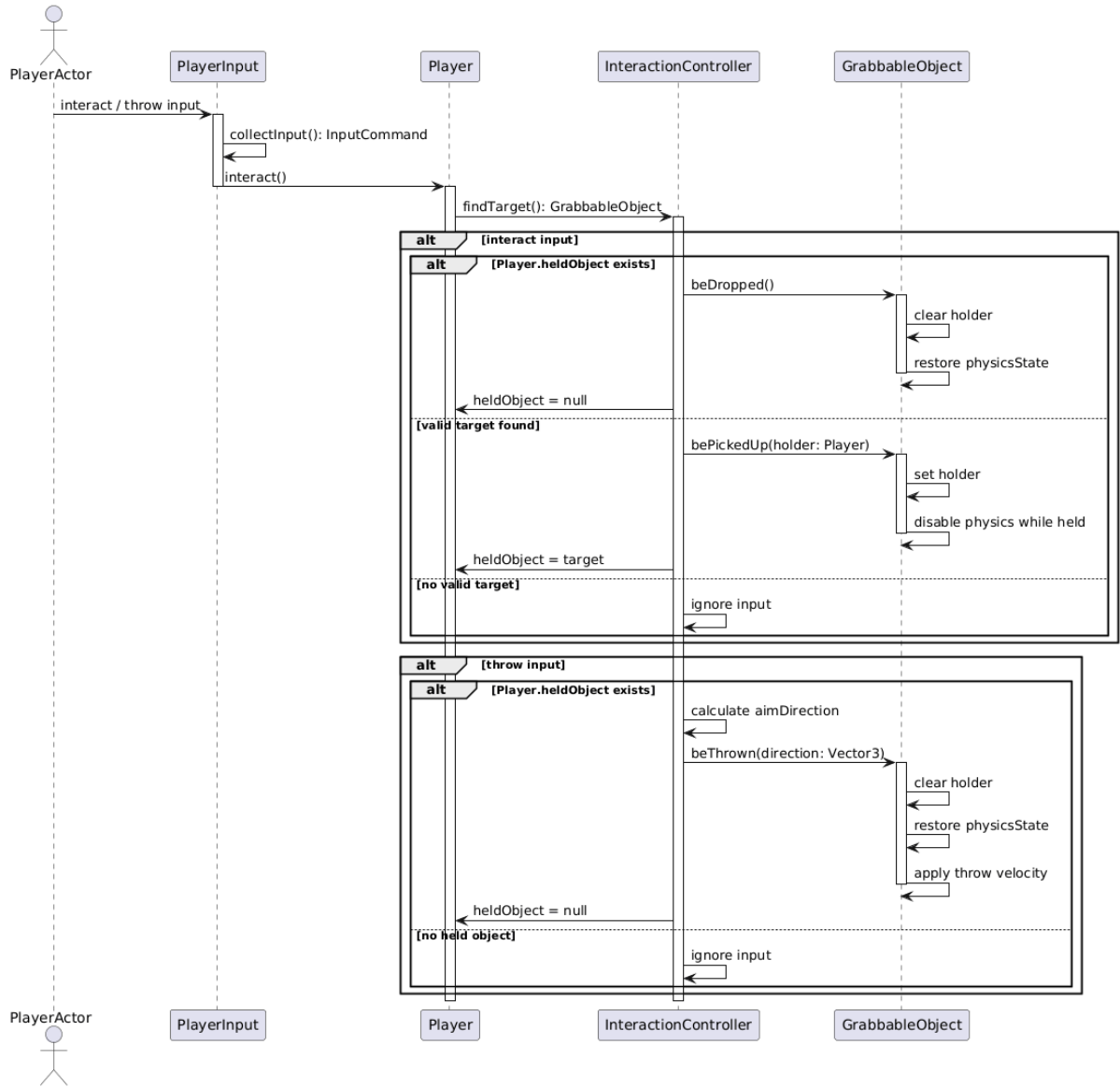
UseCase 1: 플레이어 이동 및 시점 조작

Use Case 1 - Player Movement and View Control



UseCase 2: 물리 오브젝트 잡기/놓기/던지기

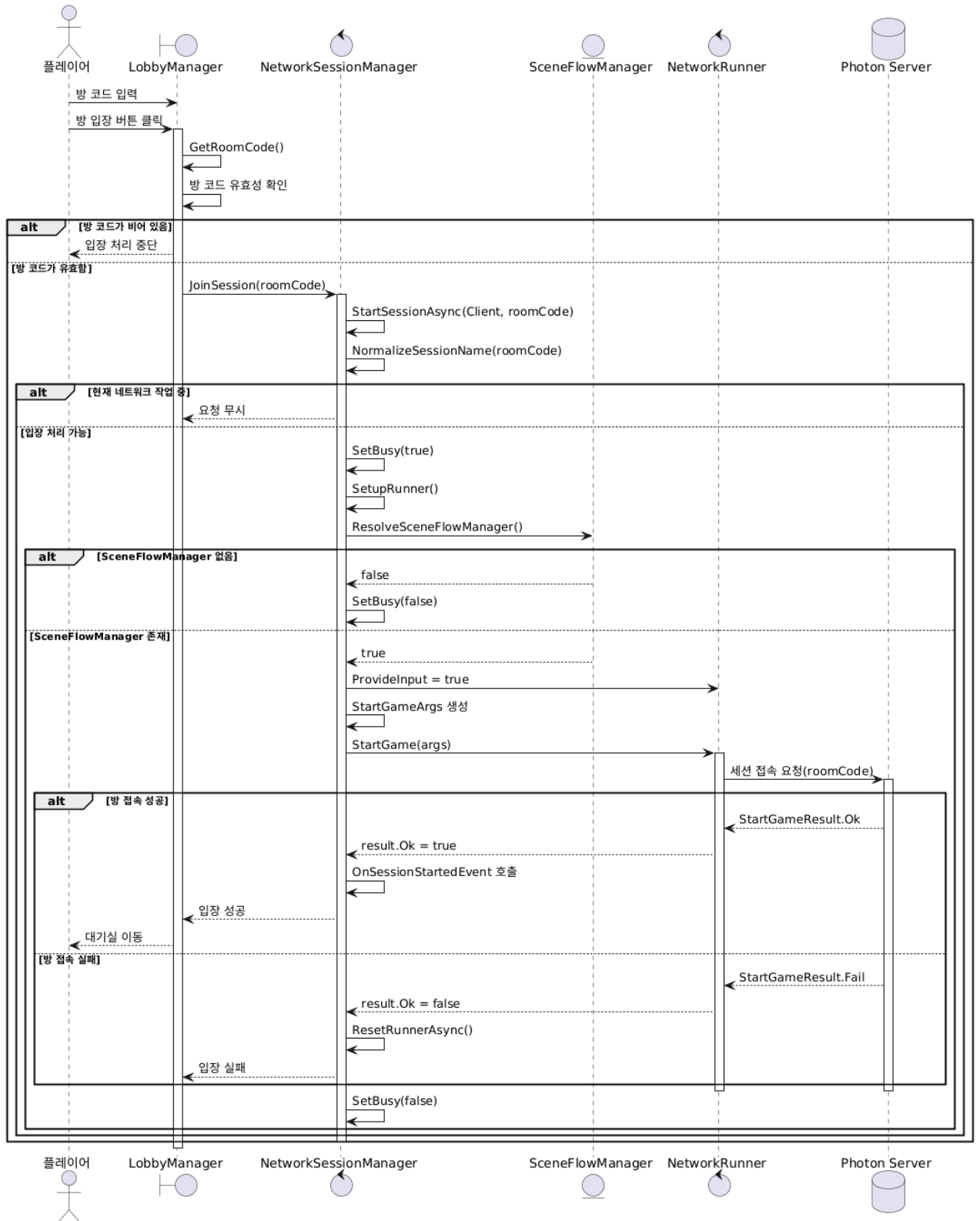
Use Case 2 - Grabbable Object Pickup / Drop / Throw



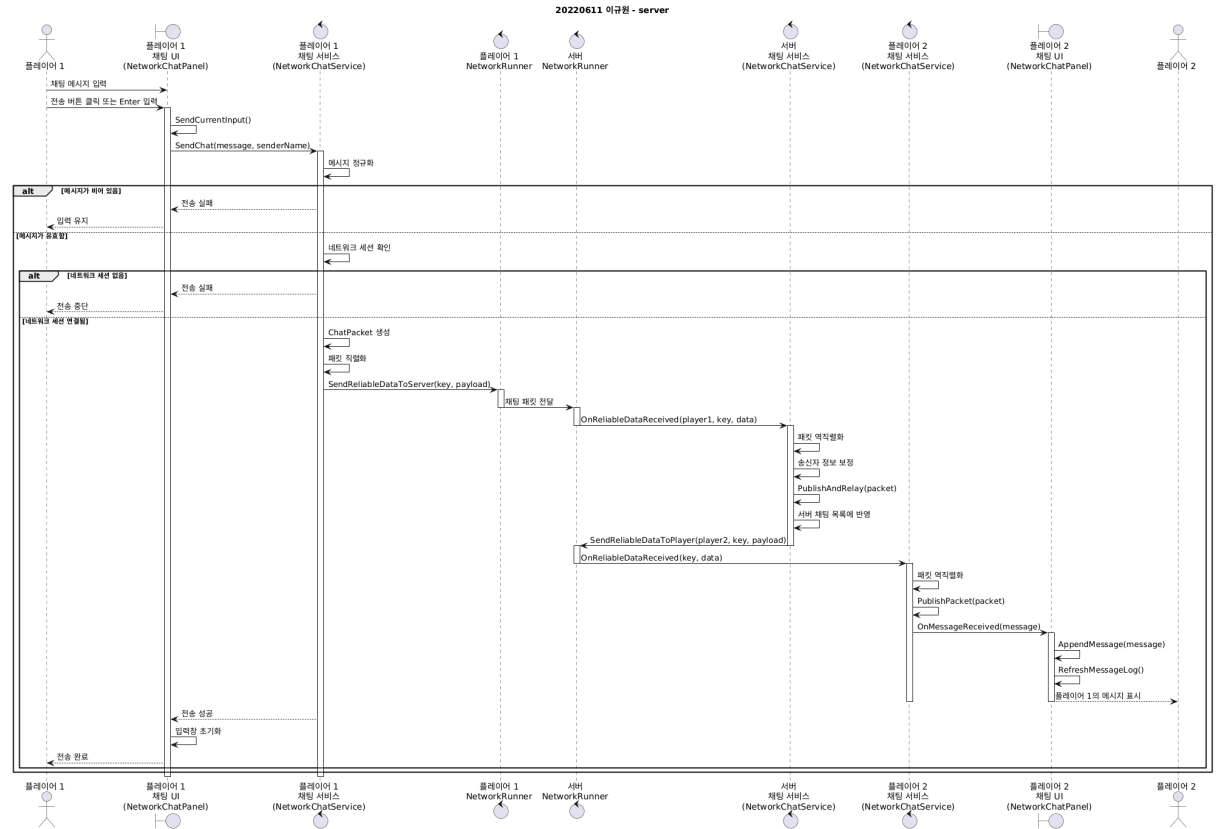
Server

UseCase 1: 방 입장하기

20220611 이규원 - server

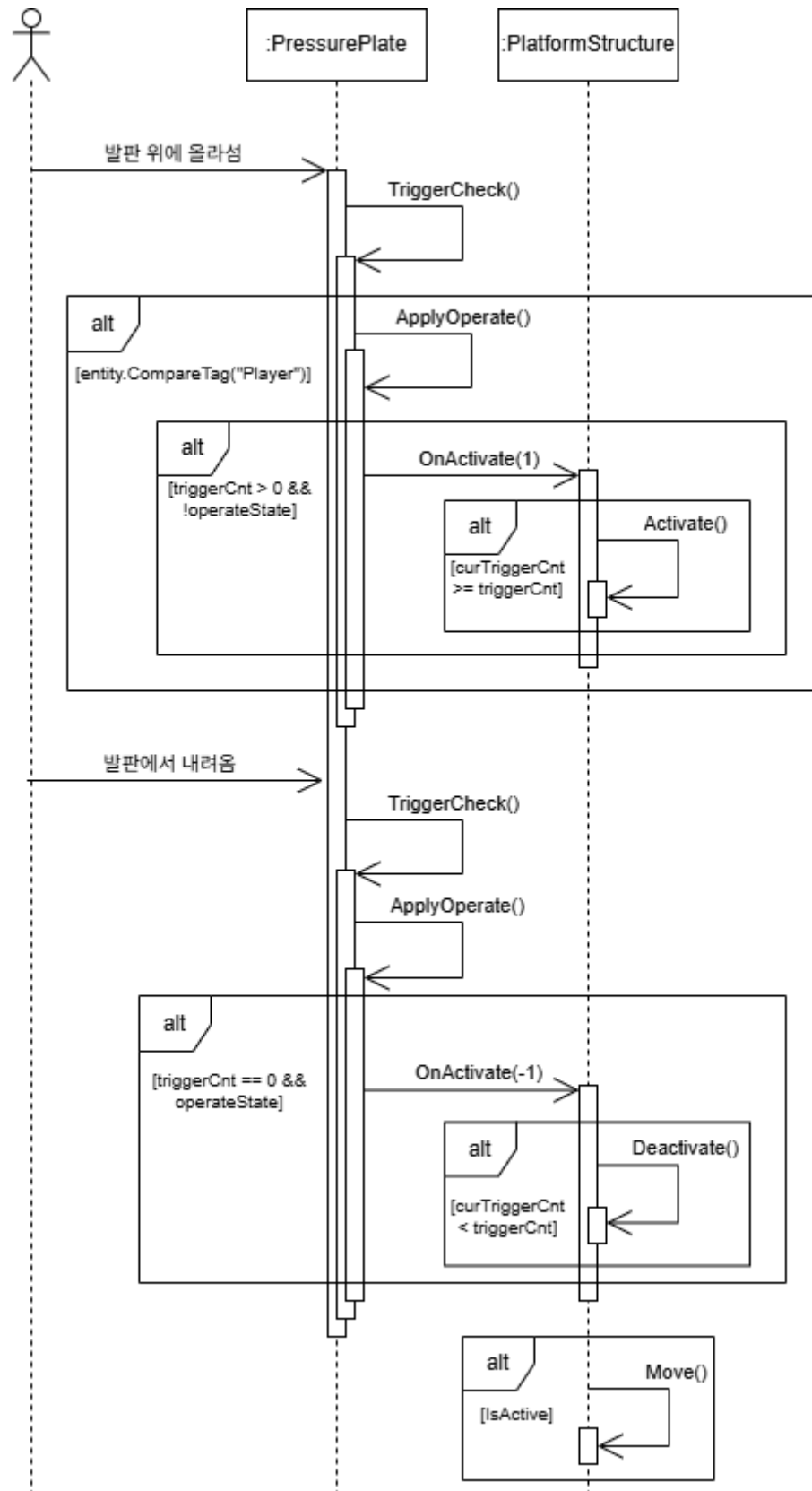


UseCase 2: 채팅 하기

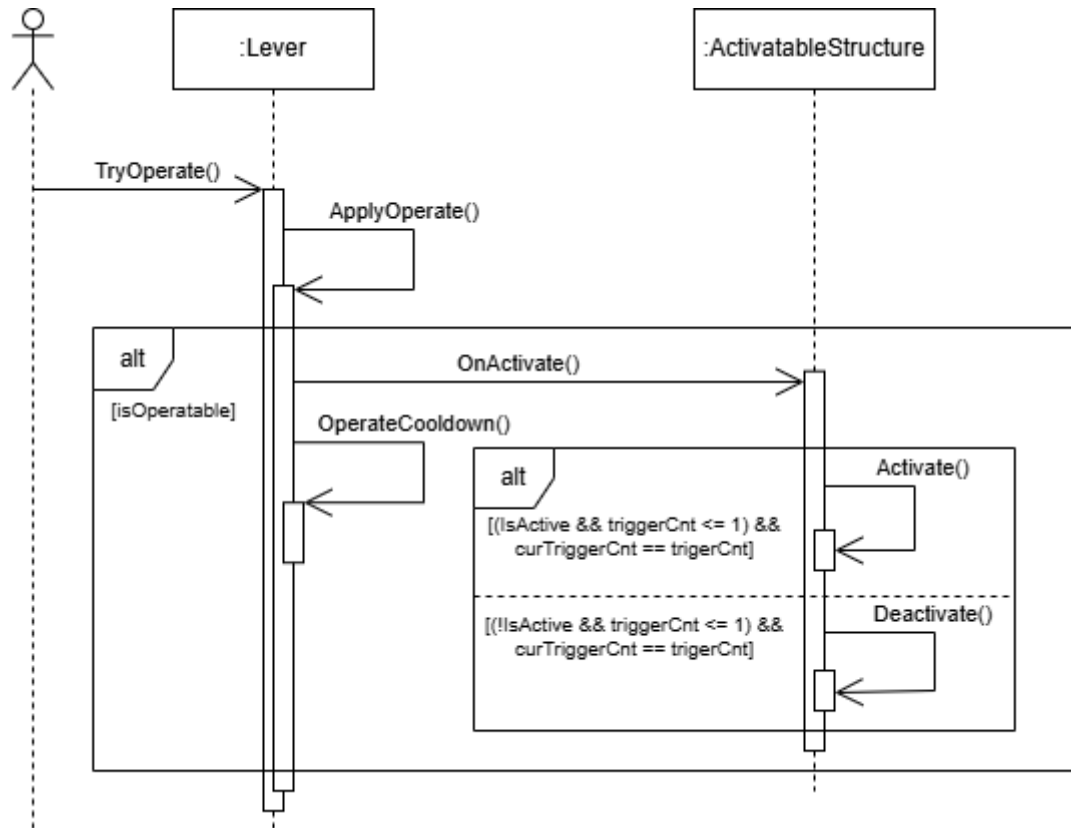


Object

UseCase 1: 레버 작동하기

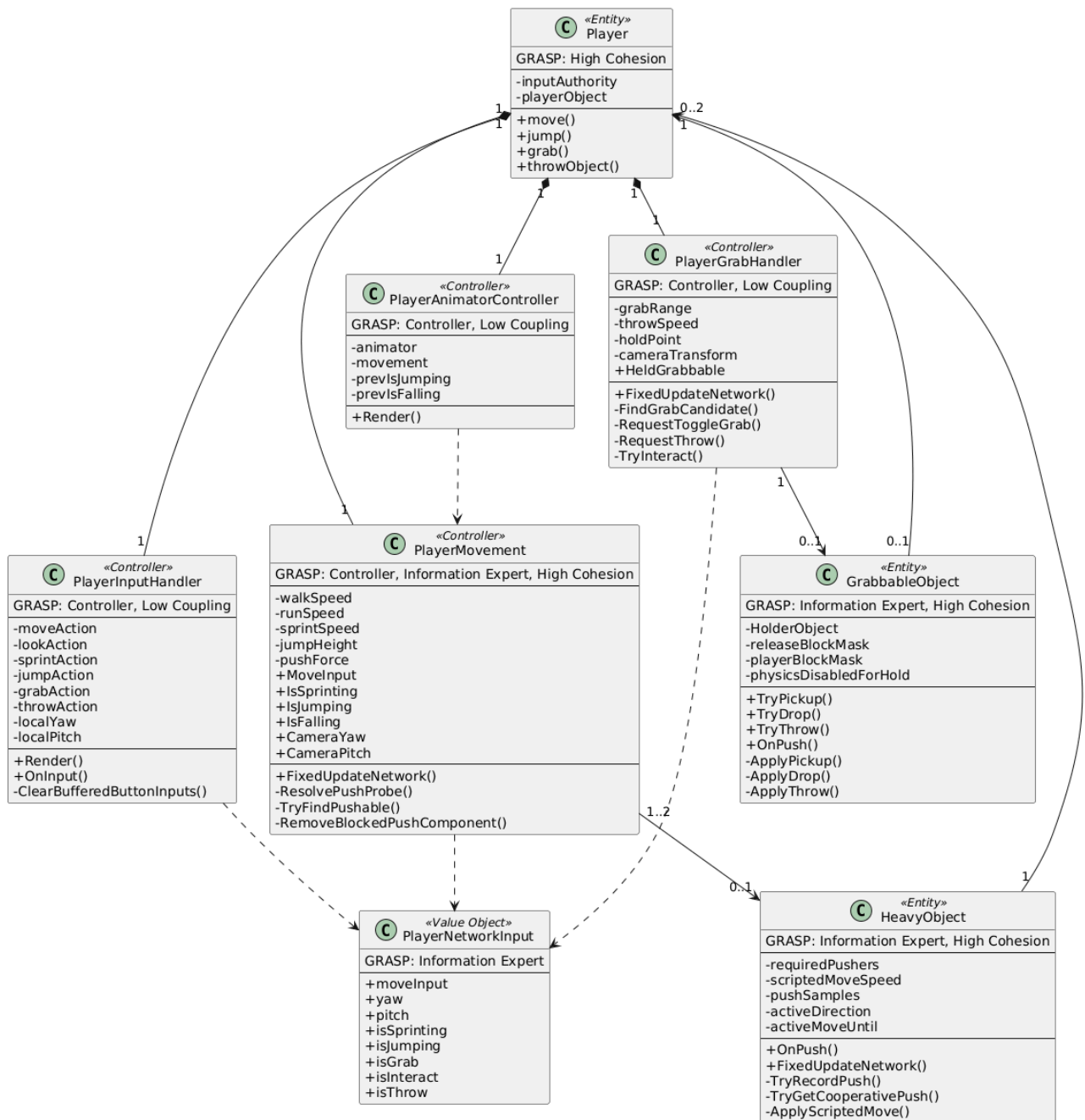


UseCase 2: 발판 작동하기



8. Class Diagram Player

Player Class Diagram



Conceptual class diagram과 차이점

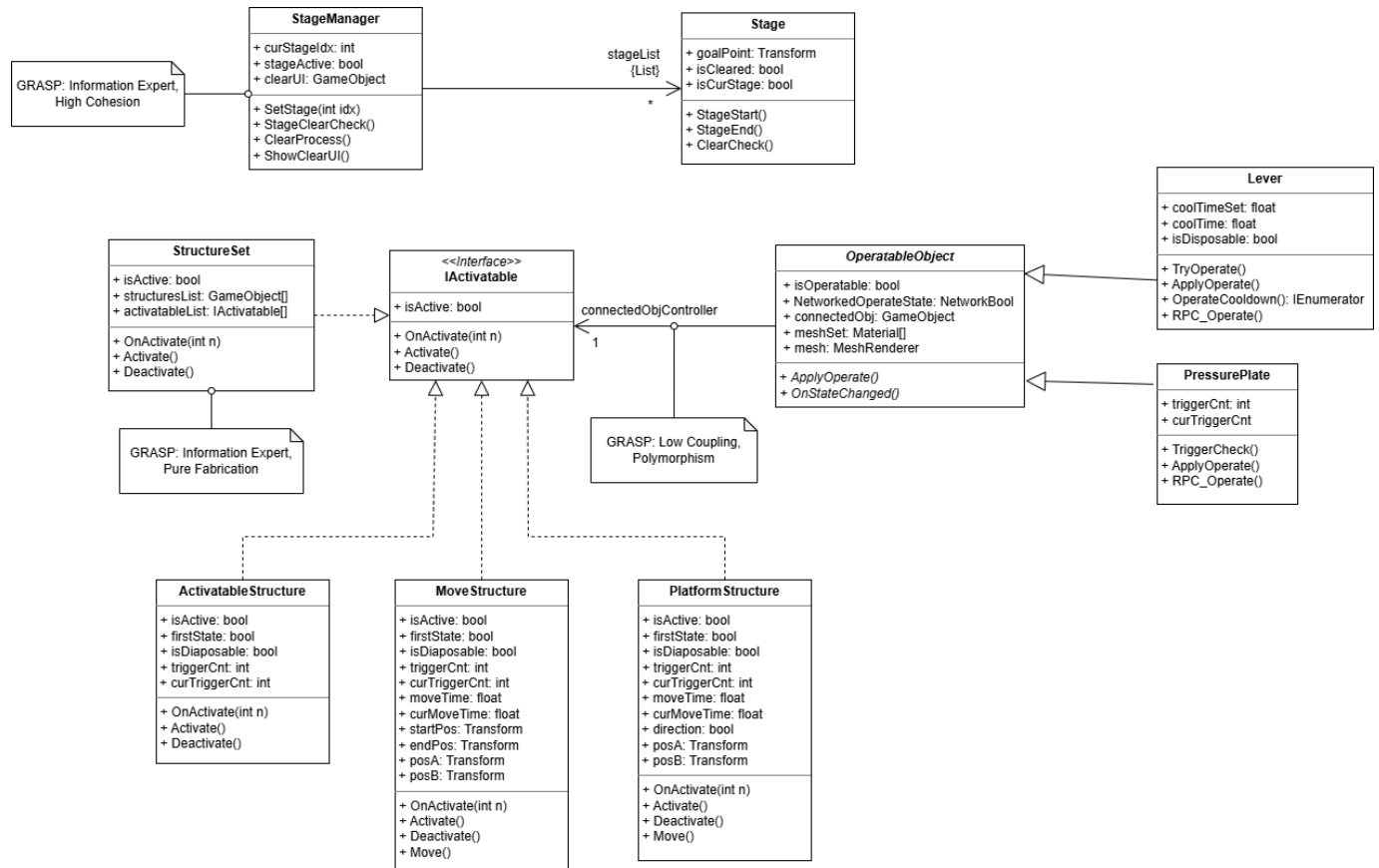
- PlayerInput이 PlayerInputHandler와 PlayerNetworkInput으로 나뉨
- MovementController를 PlayerMovement로 구체화하면서 FixedUpdateNetwork(), ResolvePushProbe() 같은 실제 네트워크 업데이트 메서드를 추가함
- PlayerNetworkInput이 추가되어 네트워크 입력 값을 별도 Value Object로 분리

20220611 이규원 - server



- 개념적 클래스 다이어그램은 RoomManager, Room, Player, GameSession 같은 핵심 도메인 객체 중심으로 구성됨.
- 설계 클래스 다이어그램은 Unity와 Photon Fusion 구현 환경을 반영해 NetworkSessionManager, LobbyManager, SceneFlowManager, NetworkRunner, SessionInfo 중심으로 변경됨.
- 개념 설계에서는 방 목록, 참가자 목록, 준비 상태, 게임 세션 상태를 직접 만든 클래스들이 소유하고 관리함.
- 구현 설계에서는 Room이나 GameSession 같은 순수 도메인 객체를 직접 유지하지 않고 Photon의 세션 정보와 NetworkRunner 상태로 대체함.
- 사용자의 방 생성 및 참가 요청은 LobbyManager가 받고, 실제 네트워크 처리는 NetworkSessionManager에 위임함.
- 개념 설계의 RuleManager는 준비 상태와 게임 시작 가능 여부를 판단하는 단일 책임을 가짐.
- 구현 설계에서는 이 책임이 WaitingRoomManager, RoomPlayerState, NetworkGameState 등 네트워크 동기화 가능한 클래스로 분산됨.
- 이러한 구조 변화는 멀티플레이 환경에서 각 플레이어의 상태를 모든 클라이언트에 동기화해야 하기 때문임.
- 결과적으로 설계 클래스 다이어그램은 개념 모델보다 기술 의존성이 강함.
- 대신 실제 네트워크 연결, 방 생성과 참가, 씬 전환, 상태 동기화를 수행할 수 있도록 책임과 의존 관계가 더 구체화됨.

Object

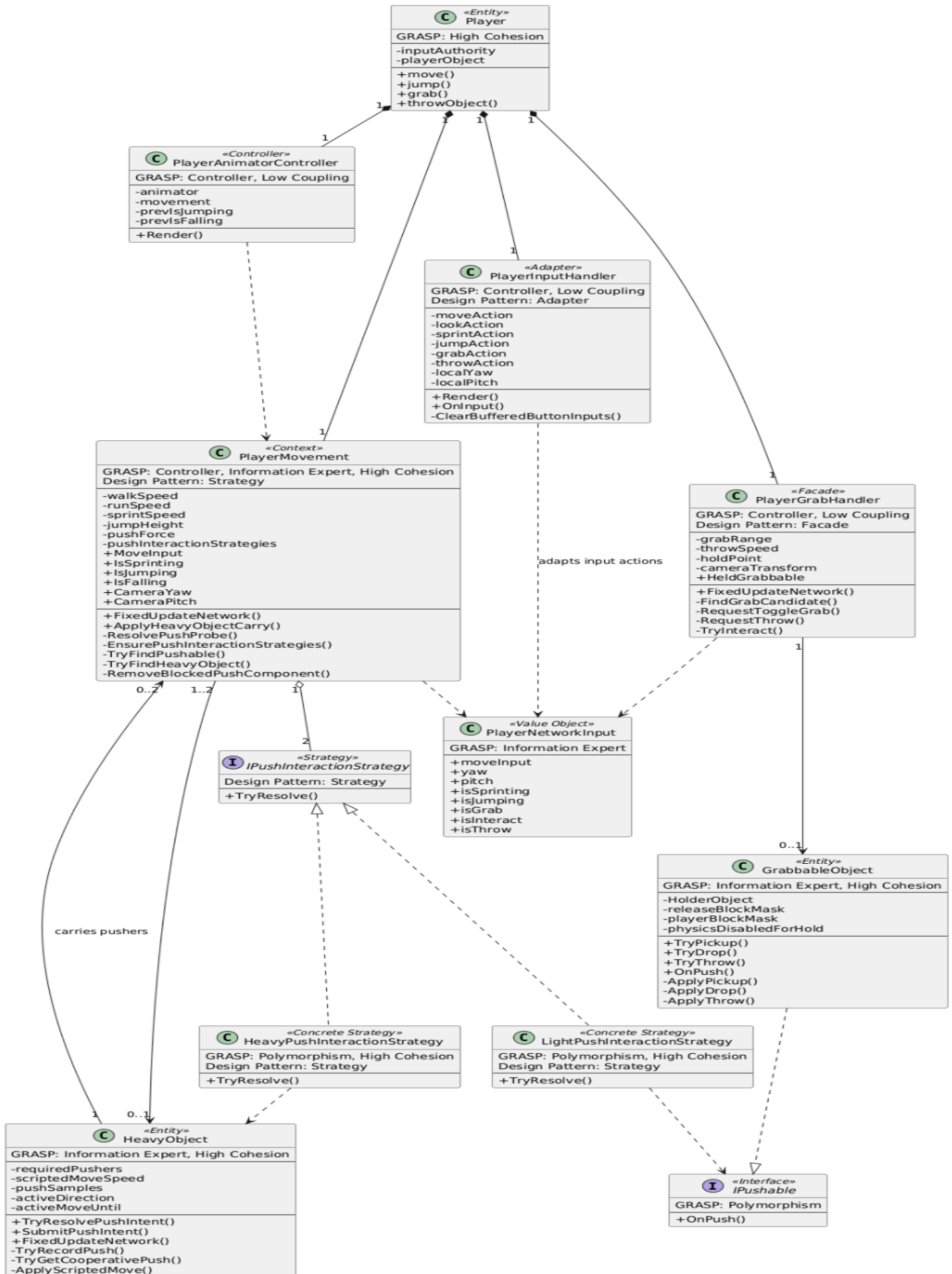


Conceptual class diagram과 차이점

- OperatableObject는 IActivatable 객체를 attribute로 가지므로 해당 인터페이스를 구현한 클래스가 아닌 인터페이스 자체와 관계를 연결하여 가시성 향상
- Button은 Lever와 가동 방식이 유사하고, 본래 기획했던 다회 상호작용 기능은 레버로도 구현 가능하다 판단, 그 역할을 레버가 흡수하고 Button 클래스는 제거
- OperatableObject와 그 관련 클래스들에 네트워크 상호작용을 위한 RPC 등 관련 변수, 메서드 추가
- OperatableObject가 여러 개의 IActivatable 객체를 제어해야 할 때, 해당 객체들을 배열로 관리하는 번거로움을 줄이고 단일 객체를 다루는 것처럼 할 수 있게 StructureSet 클래스를 추가

9. 최종 Class Diagram Player

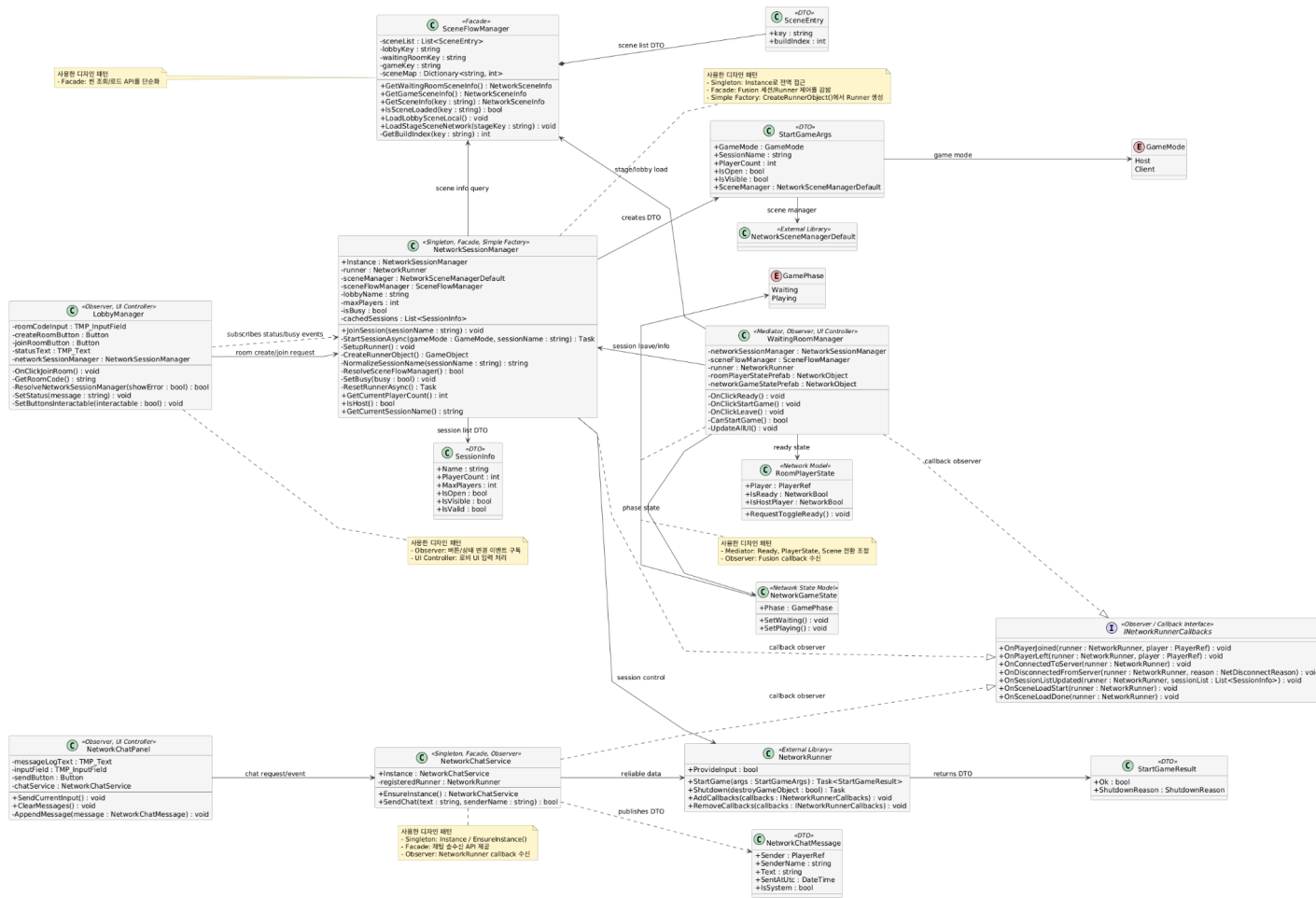
HW4 Player Class Diagram - Design Pattern Applied



Before 단계의 Class 등 설계 classifier	After 단계의 Class 등 설계 classifier(s)	적용한 설계 개념 (GRASP, Design Pattern 적용)	Architecture Design Rationale (본인이 생각하는 합리성)	NFR 와 QA 에 대한 영향 분석
PlayerMovement	PlayerMovement, IPushInteractionStrategy, HeavyPushInteractionStrategy, LightPushInteractionStrategy	GRASP: Controller, Polymorphism, High Cohesion Design Pattern: Strategy	기존 PlayerMovement가 일반 오브젝트 push와 heavy object 협동 push를 직접 분기했다. 변경 후 push 대상에 따라 달라지는 알고리즘을 strategy classifier로 분리하여 이동 흐름과 push 정책을 구분했다.	Maintainability 향상: push 정책 변경 영향 범위가 축소된다. Modifiability 향상: 새로운 push 전략 추가가 쉽다. Reliability 유지: heavy 우선, light 후순위의 기존 판정 순서를 유지한다.
PlayerInputHandler	PlayerInputHandler, PlayerNetworkInput	GRASP: Controller, Low Coupling, Information Expert Design Pattern: Adapter	Unity Input System의 입력 표현과 Fusion NetworkInput 전달 형식은 서로 다르다. PlayerInputHandler가 local input action을 PlayerNetworkInput으로 변환하여 movement/grab controller가 입력 장치 세부사항에 직접 의존하지 않게 한다.	Interoperability 향상: 입력 시스템과 네트워크 입력 구조 사이의 변환 지점이 명확하다. Testability 향상: PlayerNetworkInput 기준으로 movement/grab 동작을 확인할 수 있다.
PlayerGrabHandler	PlayerGrabHandler, GrabbableObject, HeavyObject	GRASP: Controller, Low Coupling Design Pattern: Facade	PlayerGrabHandler가 grab, drop, throw, interact 요청의 진입점 역할을 한다. 클라이언트 입력은 세부 RPC, reach validation, hold point 계산을 직접 다루지 않고 facade를 통해 요청한다.	Understandability 향상: player interaction 흐름을 한 classifier에서 추적할 수 있다. Reliability 향상: authority 검증과 거리 검증을 facade 내부에서 일관되게 수행한다.

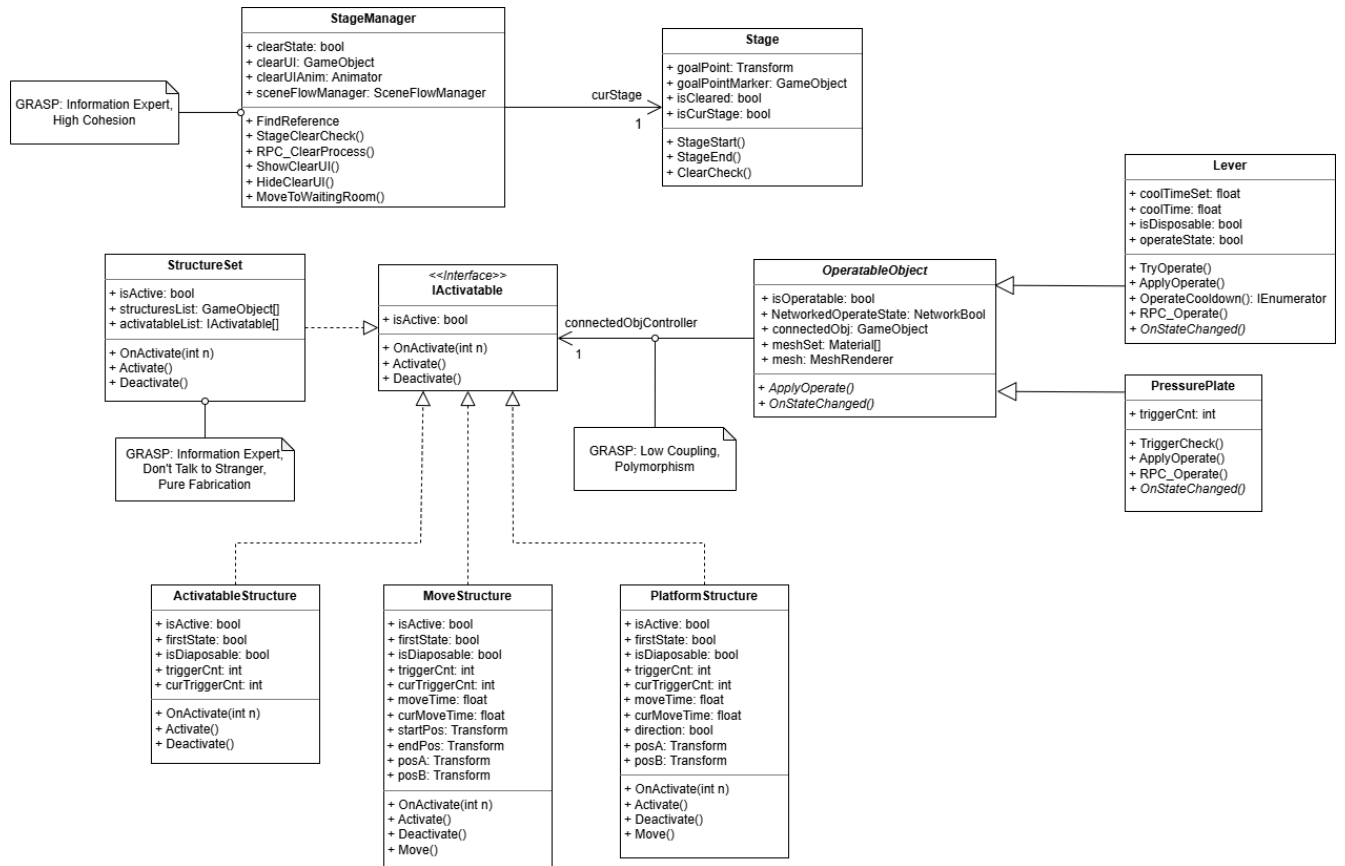
Server

20220611 이규원 - server Design Pattern class diagram



Before 단계의 Class 등 설계 classifier	After 단계의 Class 등 설계 classifier(s)	적용한 설계 개념 (GRASP, Design Pattern 적용)	Architecture Design Rationale (본인이 생각하는 합리성)	NFR 와 QA 에 대한 영향 분석
대기방에서 Ready 상태, 시작 조건, 썬 전환 요청이 분산될 수 있는 구조	WaitingRoomManager , RoomPlayerState, NetworkGameState, SceneFlowManager	GRASP: Controller, Information Expert / Pattern: Mediator, Observer, Network State Model	WaitingRoomManager가 대기방 흐름을 조정하고, 각 플레이어의 Ready 상태는 RoomPlayerState, 게임 진행 상태는 NetworkGameState가 보유한다. 상태를 가진 객체가 자신의 정보를 관리하도록 나눈다.	일관성, 신뢰성 향상. Ready 상태와 게임 Phase가 네트워크 상태 객체로 관리되어 클라이언트 간 동기화 품질이 좋아진다.
LobbyManager가 UI 입력 처리와 네트워크 세션 요청을 모두 직접 처리	LobbyManager, NetworkSessionManager	GRASP: Controller, Low Coupling / Pattern: Observer, UI Controller	LobbyManager는 버튼 클릭과 입력값 검증만 담당하고, 실제 방 생성/입장은 NetworkSessionManager에 위임한다. UI가 Photon Fusion 세부 API에 직접 의존하지 않아 구조가 단순해진다.	유지보수성 향상. UI 변경과 네트워크 로직 변경이 분리된다. 버튼 비활성화/상태 메시지 처리로 사용성도 좋아진다.
채팅 UI가 네트워크 송수신 세부 로직까지 직접 처리할 수 있는 구조	NetworkChatPanel, NetworkChatService, NetworkChatMessage	GRASP: Low Coupling, High Cohesion / Pattern: Singleton, Facade, Observer	NetworkChatPanel은 입력과 출력 UI만 담당하고, NetworkChatService가 메시지 송수신, 직렬화, 중계를 담당한다. 메시지는 NetworkChatMessage 로 전달한다.	유지보수성, 확장성 향상. 채팅 UI를 바꾸더라도 네트워크 전송 로직은 유지할 수 있다. 메시지 중복 처리와 길이 제한으로 신뢰성도 개선된다.

Object



Before 단계의 Class 등 설계 classifier	After 단계의 Class 등 설계 classifier(s)	적용한 설계 개념 (GRASP, Design Pattern 적용)	Architecture Design Rationale (본인이 생각하는 합리성)	NFR 와 QA 에 대한 영향 분석
Stage	Stage, StageManager	Information Expert, High Cohesion	Stage에서 스테이지의 시작과 진행, 클리어 조건 확인과 클리어 프로세스를 전담하지 않고 StageManager를 두어 역할을 분리하였다. Stage는 자신의 클리어 상태 검사만 수행하고, Manager가 그걸 받아 클리어 프로세스를 진행하거나 씬이 시작될 때 스테이지를 설정하도록 한다.	Faultlessness: 클리어 조건으로 설정한 모브젝트가 작동했을 때만 정확히 클리어 프로세스를 실행한다.
IActivatable	IActivatable	Polymorphism, Low Coupling	OperatableObject가 여러 종류의 구조물과 간단하게 상호작용할 수 있게 할 수 있게 한다. 각 구조물별로 다른 상호작용 방식을 OperatableObject가 직접 다루지 않고, IActivatable을 구현한 구조물들을 통해 동일한 함수로 각기 다른 상호작용 방식이 실현되도록 한다.	Modularity: 새로운 형태의 구조물을 구현할 때, OperatableObject 클래스의 코드를 수정하지 않고도 인터페이스 구현을 통해 구조물을 추가할 수 있다.
StructureSet	StructureSet	Don't Talk to Stranger, Pure Fabrication	하나의 OperatableObject가 여러 개의 구조물을 다루어야 할 때, 모든 구조물에 일일이 접근해서 작동 함수를 호출하지 않고, 그 역할을 해 줄 클래스를 만든다. IActivatable을 구현하여 다른 구조물과 같은 방식으로 호출, 호출 시 자신에 포함된 모든 구조물의 작동 함수를 호출하도록 한다.	Functional Correctness: 여러 구조물을 복잡한 로직 없이 중간 매개 구조물을 넣어 간편하게 호출할 수 있도록 한다. 여러 구조물을 정확하게 제어할 수 있게 된다.

10. 구현 코드와 구현 결과 Player

Use Case1: 플레이어 이동 및 시점 조작

PlayerInputHandler는 로컬 플레이어 입력을 수집해 PlayerNetworkInput으로 변환한다. PlayerMovement는 네트워크 틱에서 입력을 받아 카메라 방향 기준 이동, 회전, 점프, 스프린트를 처리한다. 이를 통해 입력 응답성과 네트워크 상태 일관성을 만족한다.



걷기 / 뛰기 / 점프

PlayerInputHandler.cs

```
79 public override void Render()
80 {
81     if (!HasInputAuthority)
82         return;
83
84     if (IsGameplayInputBlocked)
85     {
86         ClearBufferedButtonInputs();
87         return;
88     }
89
90     if (jumpAction.WasPressedThisFrame())
91         localJumpPressed = true;
92
93     if (grabAction.WasPressedThisFrame())
94         localGrabPressed = true;
95
96     if (interactAction.WasPressedThisFrame())
97         localInteractPressed = true;
98
99     if (throwAction.WasPressedThisFrame())
100         localThrowPressed = true;
101
102     Vector2 lookInput = lookAction.ReadValue<Vector2>();
103     localYaw += lookInput.x * lookSensitivity;
104     localPitch -= lookInput.y * lookSensitivity;
105     localPitch = Mathf.Clamp(localPitch, minPitch, maxPitch);
106 }
```

Render(): 로컬 입력과 시점 입력 수집


```

108     public void OnInput(NetworkRunner runner, NetworkInput input)
109     {
110         if (IsGameplayInputBlocked)
111         {
112             input.Set(new PlayerNetworkInput
113             {
114                 moveInput = Vector2.zero,
115                 yaw = localYaw,
116                 pitch = localPitch,
117                 isSprinting = false,
118                 isJumping = false,
119                 isGrab = false,
120                 isInteract = false,
121                 isThrow = false
122             });
123
124             ClearBufferedButtonInputs();
125             return;
126         }
127
128         PlayerNetworkInput data = new PlayerNetworkInput
129         {
130             moveInput = moveAction.ReadValue<Vector2>(),
131             yaw = localYaw,
132             pitch = localPitch,
133             isSprinting = sprintAction.IsPressed(),
134             isJumping = localJumpPressed,
135             isGrab = localGrabPressed,
136             isInteract = localInteractPressed,
137             isThrow = localThrowPressed
138         };

```

OnInput(): 입력을 PlayerNetworkInput으로 변환

PlayerMovement.cs

```

81     public override void FixedUpdateNetwork()
82     {
83         if (!GetInput(out PlayerNetworkInput input))
84         {
85             if (Object != null && Object.HasStateAuthority)
86             {
87                 IsPushing = false;
88                 IsSprinting = false;
89                 MoveInput = Vector2.zero;
90             }
91
92             return;
93         }
94
95         CameraYaw = input.yaw;
96         CameraPitch = input.pitch;
97
98         bool jumping = input.isJumping && cc.Grounded;
99         IsJumping = jumping;
100
101         if (jumping)
102             cc.Jump(overrideImpulse: Mathf.Sqrt(jumpHeight * -2f * gravity));
103
104         Quaternion cameraYaw = Quaternion.Euler(0f, input.yaw, 0f);

```

FixedUpdateNetwork(): 이동, 회전, 점프, 등 적용

또한 밀기 로직을 **Strategy** 패턴으로 분리하여 가벼운 물체는 **IPushable**을 찾으면 힘을 전달해서 **Rigidbody**로 밀리도록 구현했다.

PlayerMovement.cs

```
355     private interface IPushInteractionStrategy
356     {
357         3 references
358         |   bool TryResolve(PlayerMovement movement, Vector3 moveDir, out PushResolution resolution);
359     }
```

```
400     private sealed class LightPushInteractionStrategy : IPushInteractionStrategy
401     {
402         2 references
403         |   public bool TryResolve(PlayerMovement movement, Vector3 moveDir, out PushResolution resolution)
404         |   {
405         |       resolution = PushResolution.PassThrough(moveDir);
406         |       if (!movement.TryFindPushable(moveDir, out _, out IPushable pushable))
407         |       |   return false;
408         |       if (movement.HasInputAuthority)
409         |       |   {
410         |       |       Vector3 force = moveDir.normalized * movement.pushForce;
411         |       |       force.y = 0f;
412         |       |       pushable.OnPush(force, movement.Object.InputAuthority);
413         |       |   }
414         |       return true;
415         |   }
416     }
```

LightPushInteractionStrategy

Use Case2: 물리 오브젝트 잡기/놓기/던지기

PlayerGrabHandler는 카메라 방향의 오브젝트를 탐색하고 **grabRange** 기준으로 상호작용 가능 여부를 검증한다. **GrabbableObject**는 잡힌 동안 물리를 비활성화하고, 놓기/던지기 시 물리 상태를 복구한다. 이를 통해 오브젝트 상호작용 정확성과 물리 상태 안정성을 만족한다.



오브젝트 조준 / 오브젝트 잡기 / 던지기

PlayerGrabHandler.cs

```
63 public override void FixedUpdateNetwork()
64 {
65     bool hasInput = GetInput(out PlayerNetworkInput input);
66     Vector3 aimDirection = hasInput ? GetAimDirection(input.yaw, input.pitch) : GetNetworkAimDirection();
67
68     if (Object.HasStateAuthority)
69         NetworkedAimDirection = aimDirection;
70
71     UpdateHoldPoint(aimDirection);
72
73     if (!HasInputAuthority || !hasInput) return;
74
75     if (input.isGrab)
76     {
77         RequestToggleGrab(FindGrabCandidate());
78     }
79 }
```

FixedUpdateNetwork(): Grab/Throw 입력 처리

```

127     private void ApplyToggleGrab(NetworkObject candidate)
128     {
129         if (HeldGrabbable != null)
130         {
131             DropHeldObject();
132             return;
133         }
134
135         if (candidate == null)
136             return;
137
138         var grabbable = candidate.GetComponent<GrabbableObject>();
139         if (grabbable == null || !CanReach(candidate))
140             return;
141
142         grabbable.TryPickup(Object);
143     }

```

잡기 검증

GrabbableObject.cs

```

70     public void TryPickup(NetworkObject holder)
71     {
72         if (Object.HasStateAuthority)
73         {
74             ApplyPickup(holder);
75             return;
76         }
77
78         RPC_Pickup(holder);
79     }

```

RPC/StateAuthority 기반 요청 처리

```

117     private void ApplyPickup(NetworkObject holder)
118     {
119         if (holder == null || HolderObject != null)
120             return;
121
122         var grabber = holder.GetComponent<PlayerGrabHandler>();
123         if (grabber == null || grabber.HeldGrabbable != null)
124             return;
125
126         HolderObject = holder;
127         grabber.HeldGrabbable = Object;
128         ClearVelocityIfDynamic();
129         SetHeldPhysicsDisabled(true);
130     }

```

실제 pickup/throw/drop 물리 상태 변경

Use Case4: 설정 메뉴 열기

StageSettingsMenu는 ESC 입력으로 설정 메뉴를 열고 닫는다. 메뉴가 열리면 PlayerInputHandler.IsGameplayInputBlocked를 활성화해 게임 입력을 차단하고, 커서와 카메라 UI 상태를 메뉴 조작에 맞게 변경한다. 메뉴를 닫거나 게임을 나갈 때 상태를 복구하여 메뉴 사용성과 입력 차단 일관성을 만족한다.



ESC키를 통한 메뉴 열기

StageSettingMenu.cs

```
113     private void Update()  
114     {  
115         if (isLeaving)  
116             return;  
117  
118         Keyboard keyboard = Keyboard.current;  
119         if (keyboard == null || !keyboard.escapeKey.wasPressedThisFrame)  
120             return;  
121  
122         if (isOpen)  
123             CloseMenu();  
124         else  
125             OpenMenu();  
126     }
```

ESC 입력으로 메뉴 토글

```

223 private void OpenMenu()
224 {
225     if (menuRoot == null || isOpen)
226         return;
227
228     isOpen = true;
229     CaptureGameplayInput();
230     menuRoot.SetActive(true);
231
232     if (EventSystem.current != null && closeButton != null)
233         EventSystem.current.SetSelectedGameObject(closeButton.gameObject);
234 }

```

메뉴 열기

```

246 private void CaptureGameplayInput()
247 {
248     previousCursorLockState = Cursor.lockState;
249     previousCursorVisible = Cursor.visible;
250     PlayerInputHandler.IsGameplayInputBlocked = true;
251
252     cachedPlayerInput = ResolveLocalPlayerInput();
253     hadPlayerInput = cachedPlayerInput != null;
254
255     if (cachedPlayerInput != null)
256     {
257         previousPlayerInputEnabled = cachedPlayerInput.enabled;
258         cachedPlayerInput.enabled = false;
259     }
260
261     cachedCamera = ResolveLocalCamera();
262     hadCamera = cachedCamera != null;
263
264     if (cachedCamera != null)
265     {
266         previousCrosshairVisible = cachedCamera.showCrosshair;
267         previousSplitSeparatorVisible = cachedCamera.showSplitSeparator;
268         previousSplitSeparatorDimmed = cachedCamera.IsSplitSeparatorDimmed;
269         cachedCamera.showCrosshair = false;
270         cachedCamera.showSplitSeparator = true;
271         cachedCamera.SetSplitSeparatorDimmed(true);
272     }
273
274     Cursor.lockState = CursorLockMode.None;
275     Cursor.visible = true;
276 }

```

입력 차단, 커서/크로스헤어 상태 저장

Server

1. Use Case: 방 입장하기

주요 구현 코드 설명

방 입장하기 기능은 LobbyManager, NetworkSessionManager, SceneFlowManager를 중심으로 구현하였다.

```
125 private void OnClickJoinRoom()  
126 {  
127     Debug.Log("[LobbyManager] Join Room button clicked");  
128  
129     if (!ResolveNetworkSessionManager(true))  
130         return;  
131  
132     string roomCode = GetRoomCode();  
133  
134     if (string.IsNullOrEmpty(roomCode))  
135     {  
136         SetStatus("Please enter a room code to join");  
137         Debug.LogWarning("[LobbyManager] Room code input is empty");  
138         return;  
139     }  
140  
141     Debug.Log("[LobbyManager] Trying to join room: " + roomCode);  
142  
143     networkSessionManager.JoinSession(roomCode);  
144 }  
145
```

-LobbyManager의 요청을 보내는 코드-

```
234 private async Task StartSessionAsync(GameMode gameMode, string sessionName)  
235 {  
236     if (isBusy)  
237         return;  
238  
239     sessionName = NormalizeSessionName(sessionName);  
240  
241     if (string.IsNullOrEmpty(sessionName))  
242     {  
243         SetStatus("Room code is empty");  
244         return;  
245     }  
246  
247     SetBusy(true);  
248     int configuredMaxPlayers = GetConfiguredMaxPlayers();  
249  
250     Debug.Log($"[NetworkSessionManager][Diagnostics:StartSessionBegin] mode={gameMode}  
251  
252     try  
253     {  
254         SetupRunner();  
255  
256         if (!ResolveSceneFlowManager())  
257         {  
258             SetStatus("SceneFlowManager not found");  
259             return;  
260         }  
261     }  
262 }
```

-networkSessionManager의 세션 참가 코드(너무 길어서 일부분 발췌)-

LobbyManager는 사용자가 로비 화면에서 입력한 방 코드를 읽고, 입력값이 비어 있는지 검사한 뒤 NetworkSessionManager에게 방 입장을 요청한다. LobbyManager의

OnClickJoinRoom()에서는 GetRoomCode()를 통해 입력된 방 코드를 가져오고, 공백이면 상태 메시지를 출력한다. 정상적인 방 코드가 입력되면 networkSessionManager.JoinSession(roomCode)를 호출하여 실제 네트워크 세션 입장 처리를 위임한다.

실제 Photon Fusion 세션 접속은 NetworkSessionManager에서 담당한다. JoinSession()은 내부적으로 StartSessionAsync(GameMode.Client, sessionName)을 호출한다. 이 메서드에서는 방 이름을 정규화하고, NetworkRunner를 준비한 뒤 StartGameArgs를 생성하여 runner.StartGame(args)를 실행한다. 클라이언트가 없는 방을 실수로 새로 만들지 않도록 EnableClientSessionCreation = false로 설정하여, 잘못된 방 코드로 새로운 방이 생성되는 문제를 방지하였다.

```
58 public NetworkSceneInfo GetWaitingRoomSceneInfo()  
59 {  
60     return GetSceneInfo(waitingRoomKey);  
61 }  
62
```

-씬을 전환해주는 코드-

또한 씬 정보는 SceneFlowManager가 관리하도록 분리하였다. NetworkSessionManager는 직접 build index를 다루지 않고 SceneFlowManager를 통해 대기방 씬 정보를 얻는다. 이를 통해 네트워크 세션 로직과 씬 관리 로직을 분리하였다.

적용한 설계 개념

LobbyManager: UI Controller, Observer

NetworkSessionManager: Singleton, Facade, Simple Factory

SceneFlowManager: Facade

구현 결과 화면 설명

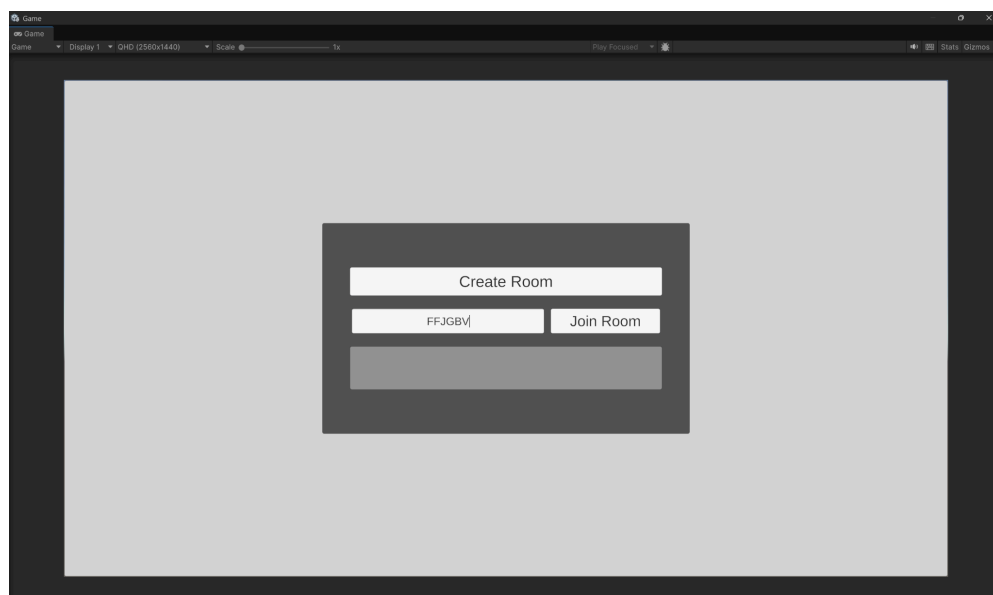


그림 1. 로비 화면에서 방 코드 입력

사용자는 로비 화면에서 방 코드를 입력하고 Join Room 버튼을 클릭한다. 이때 입력된 방 코드는 LobbyManager에서 검사되며, 빈 값이면 안내 메시지를 출력하고 입장을 중단한다.

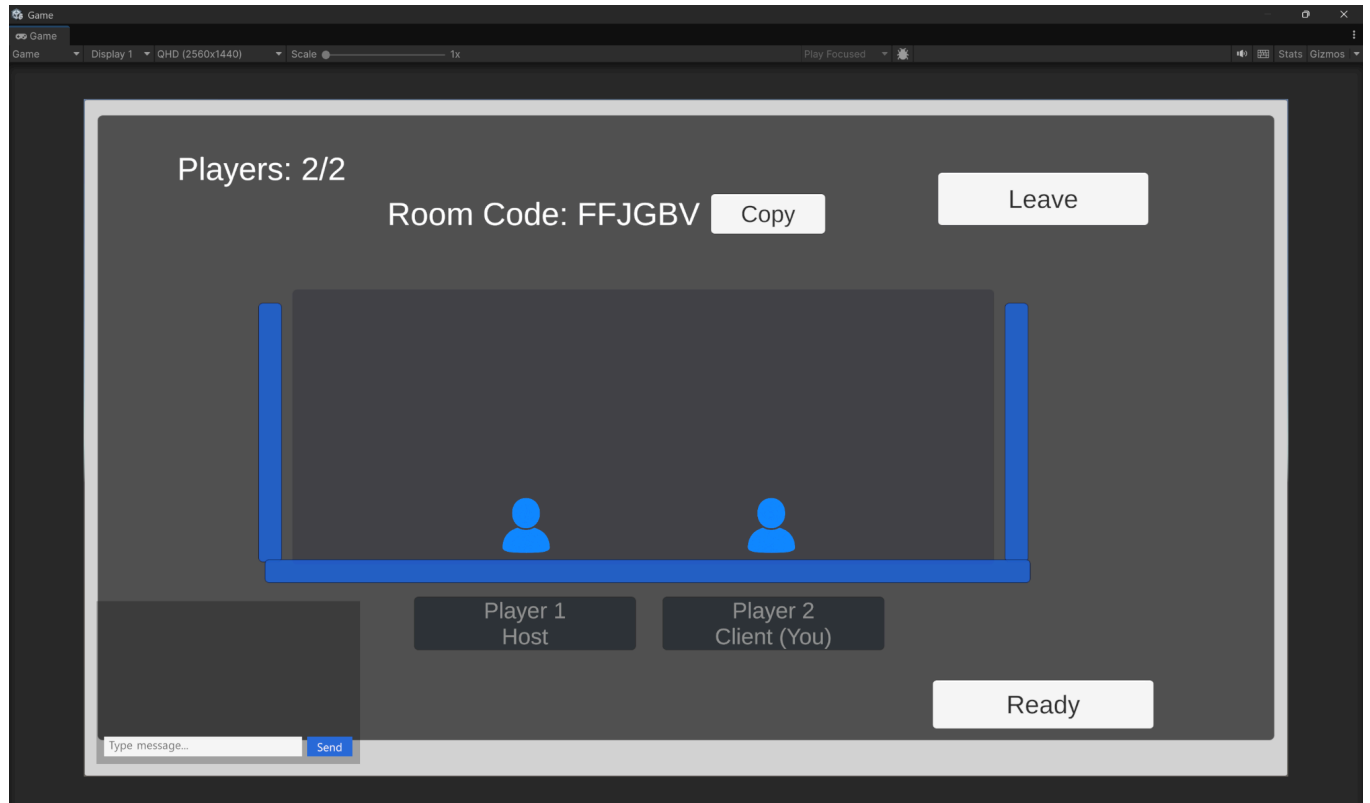


그림 2. 방 입장 성공 후 대기방 화면

정상적인 방 코드가 입력되면 **NetworkSessionManager**가 **Photon Fusion** 세션에 **Client** 모드로 접속한다. 입장에 성공하면 대기방 씬으로 이동하고, 현재 방 코드와 접속한 플레이어 수가 화면에 표시된다. 이후 사용자는 **Ready** 버튼을 눌러 게임 시작 준비 상태를 변경할 수 있다.

2. Use Case: 채팅하기

주요 구현 코드 설명

채팅하기 기능은 **NetworkChatPanel**, **NetworkChatService**, **NetworkChatMessage**를 중심으로 구현하였다.

```

public bool SendChat(string text, string senderName = null)
{
    text = NormalizeText(text, maxMessageLength);

    if (string.IsNullOrEmpty(text))
        return false;

    if (!TryGetRunner(out NetworkRunner activeRunner) || !activeRunner.IsRunning)
    {
        PublishSystemMessage("Chat is not connected to a network session.");
        return false;
    }

    PlayerRef sender = activeRunner.LocalPlayer;
    int sequence = ++localSequence;
    string normalizedSenderName = NormalizeSenderName(activeRunner, senderName, sender);

    ChatPacket packet = new ChatPacket
    {
        PacketType = activeRunner.IsServer ? PacketServerBroadcast : PacketClientToServer,
        SenderRaw = sender.RawEncoded,
        Sequence = sequence,
        SenderName = normalizedSenderName,
        Text = text,
        SentAtUtcTicks = DateTime.UtcNow.Ticks
    };

    if (activeRunner.IsServer)
    {
        PublishAndRelay(activeRunner, packet);
        return true;
    }

    byte[] payload = SerializePacket(packet);

    if (payload.Length > MaxMessageBytes)
    {
        PublishSystemMessage("Chat message is too long.");
        return false;
    }

    activeRunner.SendReliableDataToServer(CreateReliableKey(packet), payload);
    return true;
}

```

- NetworkChatService.SendChat() -

NetworkChatPanel은 채팅 UI를 담당하며, 사용자가 입력창에 메시지를 입력하고 전송 버튼을 누르면 SendCurrentInput()이 실행된다. 이 메서드는 입력된 문자열을 가져온 뒤 NetworkChatService.SendChat()을 호출하여 실제 네트워크 전송을 위임한다.

NetworkChatService는 채팅 메시지 송수신을 담당하는 서비스 클래스이다. 메시지를 보낼 때는 먼저 공백 제거, 줄바꿈 제거, 최대 길이 제한 등의 정규화 과정을 거친다. 이후 현재 NetworkRunner가 실행 중인지 확인하고, 서버인지 클라이언트인지에 따라 다른 방식으로 메시지를 처리한다.

Host인 경우에는 메시지를 바로 **PublishAndRelay()**를 통해 다른 플레이어에게 중계한다. Client인 경우에는 **SendReliableDataToServer()**를 이용해 서버로 메시지를 전송한다. 서버는 수신한 메시지를 다시 모든 플레이어에게 전달한다.

수신된 메시지는 **OnMessageReceived** 이벤트를 통해 **NetworkChatPanel**에 전달된다. **NetworkChatPanel**은 이 이벤트를 구독하고 있다가 메시지를 받으면 화면의 채팅 로그에 내용을 추가한다.

적용한 설계 개념

NetworkChatPanel: UI Controller, Observer

NetworkChatService: Singleton, Facade, Observer

INetworkRunnerCallbacks: Observer / Callback Interface

구현 결과 화면 설명

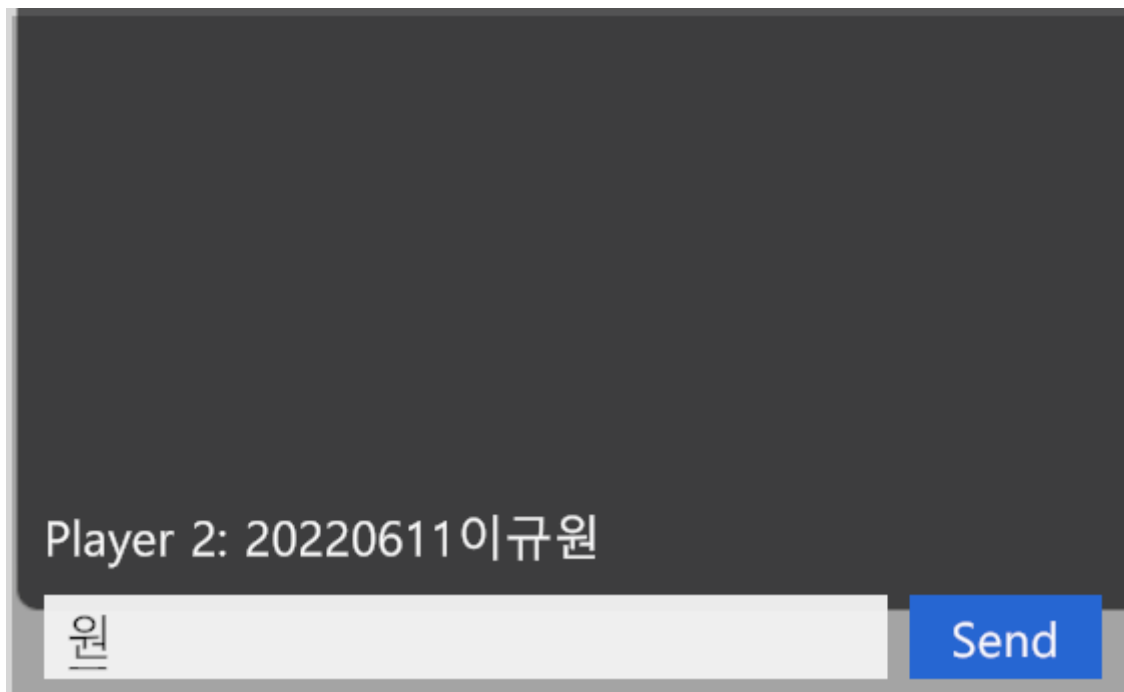


그림 3. 대기방의 채팅 패널

사용자는 채팅 입력창에 메시지를 입력하고 전송 버튼을 클릭한다. 입력된 메시지는 **NetworkChatPanel**에서 처리된 뒤 **NetworkChatService**로 전달된다. UI 클래스가 직접 네트워크 전송을 처리하지 않고 서비스 클래스에 위임하도록 구현하였다.

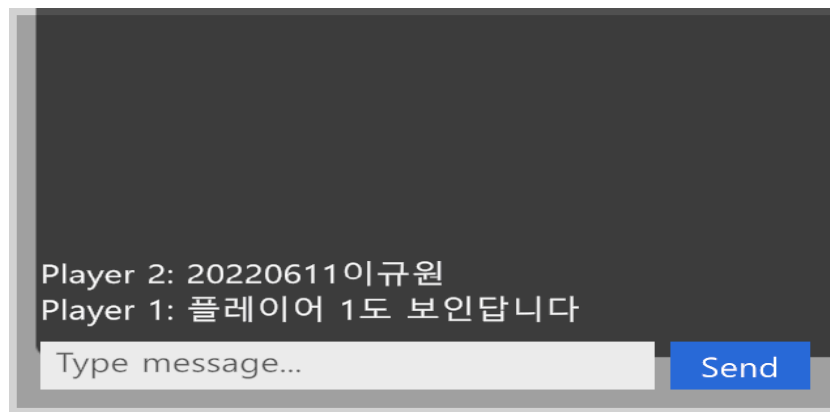


그림 4. 다른 플레이어에게 채팅 메시지 동기화

Host 또는 **Client**가 보낸 메시지는 **Photon Fusion**의 **Reliable Data** 기능을 통해 네트워크로 전달된다. 수신한 메시지는 **OnMessageReceived** 이벤트를 통해 각 클라이언트의 채팅 패널에 표시된다. 이를 통해 여러 플레이어가 같은 채팅 내용을 실시간으로 확인할 수 있다.

Object

Use Case2: 기믹 수행 - 레버 작동

플레이어가 레버와 상호작용하면 레버의 TryOperate()가 호출되어 상호작용을 실시한다. 자신과 연결된 IActivatable 오브젝트(이하 구조물)의 OnActivate를 호출, 구조물이 활성화 상태로 변경되어 숨김 상태에서 모습을 보이거나 플레이어가 이용할 수 있는 위치로 이동한다.

상호작용을 마친 뒤, 레버의 일회성 여부를 판단하는 isDisposable 변수가 true이기에 레버는 비활성 상태의 material로 색을 바꾸고, 더 이상 상호작용을 받지 않도록 차단한다.

Lever.cs

```
public void TryOperate(NetworkObject operatorObject)
{
    RPC_Operate(operatorObject);
}

protected override void ApplyOperate(NetworkObject operatorObject)
{
    if(operatorObject == null)
    {
        return;
    }

    if(!isOperatable)
    {
        return;
    }

    OperateState = !OperateState;
    operateState = !operateState;
    connectedObjController.OnActivate(operateState ? 1 : -1);
    StartCoroutine(OperateCooldown());
    // OnStateChanged();
}

[Rpc(RpcSources.All, RpcTargets.All)]
void RPC_Operate(NetworkObject operatorObject) => ApplyOperate(operatorObject);
```

MoveableStructure.cs

```
public void OnActivate(int n)
{
    if(triggerCnt <= 1)
    {
        if(!IsActive)
            Activate();
        else
            Deactivate();
        if (isDisposable)
            triggerCnt = 100;
    }
    else if(triggerCnt == 100)
        return;
    else
    {
        curTriggerCnt += n;
        if(curTriggerCnt >= triggerCnt)
        {
            Activate();
            if (isDisposable)
                triggerCnt = 100;
        }
        else
            Deactivate();
    }
}
```

```
public void Activate()
{
    IsActive = true;
    startPos = transform.position;
    endPos = posB.position;

    if (coroutine != null) StopCoroutine(coroutine);
    currentT = 0f;
    coroutine = StartCoroutine(Move());
}

public void Deactivate()
{
    IsActive = false;
    startPos = transform.position;
    endPos = posA.position;

    if (coroutine != null) StopCoroutine(coroutine);
    currentT = 0f;
    coroutine = StartCoroutine(Move());
}
```



레버 작동 전과 후

Use Case4: 최종 기믹 해결 및 스테이지 클리어

두 플레이어가 기믹을 해결하고 도착 지점에 도달한다. 빛 기둥으로 표현된 구역에 두 플레이어가 동시에 들어와 있으면 빛 기둥이 사라지고 클리어를 알리는 UI가 나타난다. 이후 잠시 뒤에 두 플레이어는 waiting room scene으로 이동한다

Stage.cs

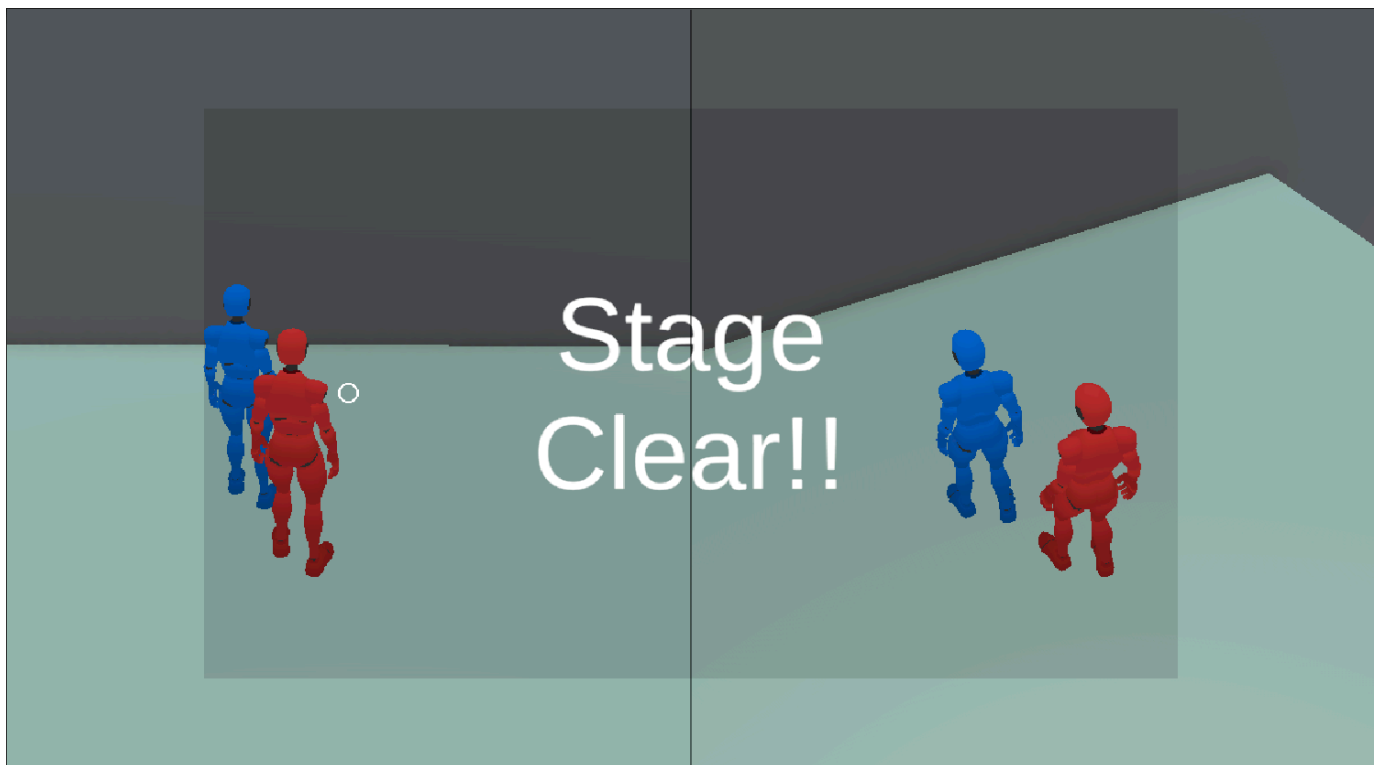
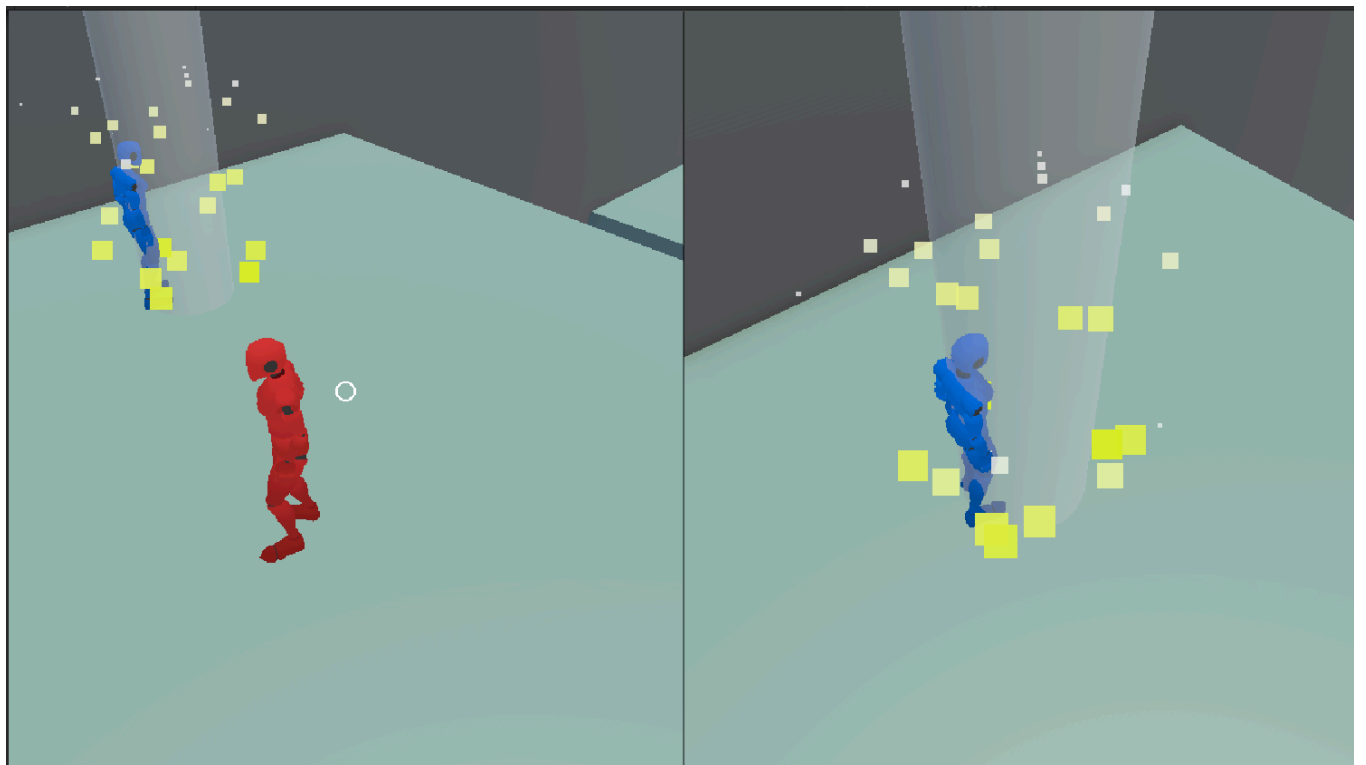
```
void ClearCheck()
{
    int cnt = 0;
    var hits = Physics.OverlapSphere(goalPoint.position, 1.5f, -1);
    foreach(var entity in hits)
    {
        if (entity.CompareTag("Player"))
        {
            cnt++;
        }
    }
    if(cnt == 2)
    {
        IsCleared = true;
    }
}
```

StageManager.cs

```
void StageClearCheck()
{
    if (curStage != null && curStage.IsCleared)
    {
        RPC_ClearProcess();
    }
}

[Rpc(RpcSources.All, RpcTargets.All)]
void RPC_ClearProcess()
{
    clearState = curStage.IsCleared;
    curStage.StageEnd();
    ShowClearUI();
}

void ShowClearUI()
{
    clearUI.SetActive(true);
    clearUIAnim.SetTrigger("doClear");
    Invoke("HideClearUI", 2.9f);
    Invoke("MoveToWaitingRoom", 3);
}
```



골인 구역 진입 전과 후

시연 영상

https://youtu.be/Sc_wv8XmzWc